

Real-Estate Intelligence Platform — Codebase Architecture, Design & Delivery Report

Field	Value
Project	LOCUS Analytics
Domain	Real-Estate Valuation, ROI & Market Intelligence (Pakistan)
Platform	JavaFX Desktop Application + Python ETL/ML Pipeline
Document	Codebase Report (v1.0)
Date	May 07, 2026

Member	Roll No.	Role
Fasih Ul Mubashir	24i-0517	System Engineer — Backend, DB, ETL, Deployment
Arham Manzoor	24i-0640	Lead Data Scientist — Domain Model, Services, ML
Ayaan Aman	24i-0663	Frontend Architect — JavaFX UI, Charts, Maps, PDF

0. Table of Contents

- 1. Executive Summary
- 2. Project Overview
- 3. System Architecture
- 4. Technology Stack
- 5. Repository Structure
- 6. Domain Model
- 7. Database Schema
- 8. Use Case Specifications (UCS)
- 9. System Sequence Diagrams (SSDs)
- 10. Sequence Diagrams (SDs)
- 11. Design Class Diagram
- 12. Backend — DAO & Service Layer
- 13. Frontend — JavaFX UI Layer
- 14. ETL Pipeline
- 15. Machine Learning Module
- 16. Reporting & PDF Generation
- 17. Testing Strategy
- 18. Deployment & Operations
- 19. Conclusion

1. Executive Summary

LOCUS Analytics is a desktop-grade real-estate intelligence platform designed for property analysts and administrators operating across Karachi, Lahore, and Islamabad. The system unifies four engineering concerns into a single, coherent product: a JavaFX desktop UI for analysts, a relational PostgreSQL backbone, a Python-based ETL pipeline that ingests live listings, and a Ridge-Regression valuation model written in Python and consumed in Java.

The platform supports twelve formal use cases ranging from Fair Market Value estimation (UC-1) through Investment Cluster identification (UC-9) and Admin-only operations such as ETL execution (UC-10) and System Configuration (UC-12). Each use case is backed by a dedicated service, a typed DAO layer, and a corresponding FXML screen; service-layer JUnit coverage is available for core analytical flows.

This report documents the architecture, source layout, data model, supported use cases, ML accuracy, and quality assurance strategy of the codebase as it stands at the conclusion of Phase 3.

Key Figures

Metric	Value
Total source files	156+ (Java, FXML, CSS, Python, SQL)
Java classes (main)	~70 across model, dao, service, ui, ml
Database tables	10 normalized entities + audit log + triggers
Use cases supported	12 (9 analyst-facing, 3 admin-only)
FXML screens	14 (Login, Intro, Main + 11 feature views)
Unit tests	80 test methods across 8 service test classes (service layer)
ML snapshot (latest committed)	R ² 0.857, MAE 11,299,624, RMSE 25,069,127
ETL coverage	Karachi, Lahore, Islamabad (city-locality dictionaries + ETL scraping/generation logic)

2. Project Overview

LOCUS Analytics targets a clear, underserved gap in the Pakistani real-estate market: data-driven property valuation and investment analysis. The product offers analysts a single workbench from which they can search the listing universe, compute Fair Market Value, evaluate rental yield and ROI, plot price trends, and export branded PDF reports for clients.

2.1 Product Scope

- **Analyst workflows:** FMV, Rental Yield, ROI, Search, Compare, Price Trends, Heatmap, Reports, Investment Clusters.
- **Administrator workflows:** ETL Dashboard, Manage Listings, System Configuration with audited mutations.
- **Authentication:** BCrypt-hashed credentials with role-based UI gating (admin vs analyst).
- **Persistence:** PostgreSQL primary; an Oracle-mirrored DDL is maintained for portability.
- **Intelligence:** A Ridge regression model trained on engineered locality / type features; model reload hook is implemented in the predictor.

2.2 Out of Scope

- Live mortgage simulation and bank-side affordability calculators.
- Mobile clients (iOS/Android) — the desktop scope is intentional for analyst usage.
- Cross-border listings — the dataset is geographically constrained to Pakistan.

2.3 Stakeholders

Stakeholder	Concern	Touchpoint
Property Analyst	Accurate valuation, fast comparables	FMV / Rental / ROI / Compare screens
System Administrator	Data freshness, configuration, auditability	ETL, Listings, Config screens
End Client	Trustworthy report deliverable	Branded PDF valuation report
Engineering Team	Maintainability, test coverage	Service / DAO interfaces, JUnit suite

3. System Architecture

The system follows a strict layered architecture: the JavaFX UI layer talks to a Service layer, which in turn delegates persistence to a DAO layer over HikariCP-managed JDBC connections. The Python ETL and ML subsystems are decoupled processes that communicate with the Java host through the database (for data) and a JSON model artifact (`ml/model.json`) for predictions.

3.1 Layered View

Layer	Responsibility
Presentation	JavaFX FXML views + CSS, controllers under <code>com.locus.ui.controller</code> . Charts via JavaFX Charts; maps via embedded WebView (Google Maps).
Service	Business rules, validation, orchestration. One interface per use case (e.g. <code>ValuationService</code> , <code>ROIService</code>) with production <code>Impl</code> wiring in <code>ServiceRegistry</code> ; stub implementations also exist for several services.
DAO	Per-entity CRUD with prepared statements. Runtime exceptions are used across DAO implementations; an explicit <code>DataAccessException</code> type also exists in the DAO package. Audit log is triggered on configuration mutations.
Persistence	PostgreSQL 15+ (primary) and Oracle (mirrored DDL). HikariCP pool. Triggers for <code>updated_at</code> and audit log.
Intelligence	Python Ridge regression. Trained / retrained offline. Java reads <code>model.json</code> via Jackson and reloads on file change.
ETL	Python scrapers + cleaners. Idempotent inserts via <code>url_hash</code> . Progress published to <code>etl_job</code> for live UI binding.

3.2 Cross-Cutting Concerns

- **Validation:** centralised under `com.locus.service.validation`; service methods reject invalid inputs before DAO calls.
- **Logging:** SLF4J facade with `slf4j-simple` binding for desktop output.
- **Security:** BCrypt password hashing; role-based UI menu gating; configuration mutations audited.
- **Error handling:** `ValidationException` is used for user input; persistence errors are surfaced as DAO/runtime exceptions and shown in the UI as user-friendly dialogs.
- **Performance:** connection pooling (HikariCP) and query indexes are configured in schema; performance outcomes depend on dataset/hardware and should be validated by benchmark runs.

4. Technology Stack

The stack was chosen to balance ease of distribution (single fat JAR), rich desktop UX (JavaFX), strong typing on the business layer (Java 17), and a productive ML iteration loop (Python + scikit-learn).

4.1 Runtime & Frameworks

Component	Technology	Version	Purpose
Language (App)	Java	17	Core application + business logic
UI Framework	JavaFX	21	Desktop UI, FXML, CSS, Charts, WebView
Build	Apache Maven	3.8+	Lifecycle, shade fat-JAR, javafx plugin
DB	PostgreSQL	15+	Primary OLTP store (mirrored DDL for Oracle)
DB Pool	HikariCP	5.1.0	Connection pooling
JSON	Jackson	2.17.1	Model artifact + DTO serialization
PDF	Apache PDFBox	2.0.30	Branded PDF report generation
Auth	jBCrypt	0.4	Password hashing
Logging	SLF4J + simple	2.0.13	Diagnostic logging
Test	JUnit Jupiter	5.10.2	Unit tests for service layer
ML	Python + scikit-learn	3.10+	Ridge regression training / retraining
Maps	Google Maps JS	—	Heatmap layer via JavaFX WebView

4.2 Why these choices?

- **JavaFX over Swing:** declarative FXML, modern CSS, native chart and WebView controls.
- **PostgreSQL over MySQL:** richer constraint expressivity, generated columns (`price_per_sqft`), and trigger ergonomics.
- **Ridge regression over a deep model:** interpretable coefficients and lightweight deployment through `model.json`.
- **HikariCP:** mature JDBC pool with low-overhead connection management for interactive UI workloads.
- **Maven Shade:** single executable JAR makes desktop distribution trivial.

5. Repository Structure

The repository is organised by concern, with a dominant Maven module for the JavaFX application and sibling Python modules for ETL and ML.

```

Locus-Analytics-/
├─ Schema.pdf                # Hand-drawn ERD reference
├─ domain_model.png          # UML domain model
└─ locus-analytics/
    ├─ schema.sql            # PostgreSQL DDL (canonical)
    ├─ schema_oracle.sql     # Oracle-mirrored DDL
    ├─ backup_setup.md
    ├─ scheduler_setup.md
    ├─ etl_pipeline/         # Standalone Python scraper
    │   ├─ scraper.py cleaner.py etl_main.py
    │   └─ db.py config.py view_data.py
    └─ locus-analytics/      # Maven app module
        ├─ pom.xml
        ├─ README.md
        └─ src/main/java/com/locus/
            ├─ config/        # DBConnection (HikariCP)
            ├─ dao/           # 10 DAO interfaces + impls
            ├─ model/         # POJOs + DTOs
            ├─ service/       # 14 service interfaces + impls
            ├─ ml/            # LinearRegressionPredictor
            ├─ exception/     # ValidationException
            └─ ui/            # App, controllers, scene mgmt
        ├─ src/main/resources/
            ├─ fxml/          # 14 screens (Login, Intro, Main, 11 feature)
            ├─ styles/styles.css # Brand-aware theme
            ├─ web/           # heatmap.html, clusters-map.html
            └─ config.properties # DB + API credentials
        ├─ src/test/java/com/locus/service/ # service-layer JUnit tests (8 classes)
        ├─ data/generate_seed.py # seed-data generator script
        ├─ etl/run_etl.py       # In-app ETL pipeline
        ├─ ml/                 # train_model.py, retrain_model.py, model.json
        └─ scripts/            # backup.sh / backup.bat

```

6. Domain Model

The domain is anchored by `Property`, the central aggregate around which `Valuation`, `RentalAnalysis`, `ROIAnalysis`, and `ValuationReport` are computed. Users specialise into `PropertyAnalyst` and `SystemAdministrator`, with administrators owning `ETLJob` and `SystemConfiguration`. The diagram below summarises the entities and their associations.

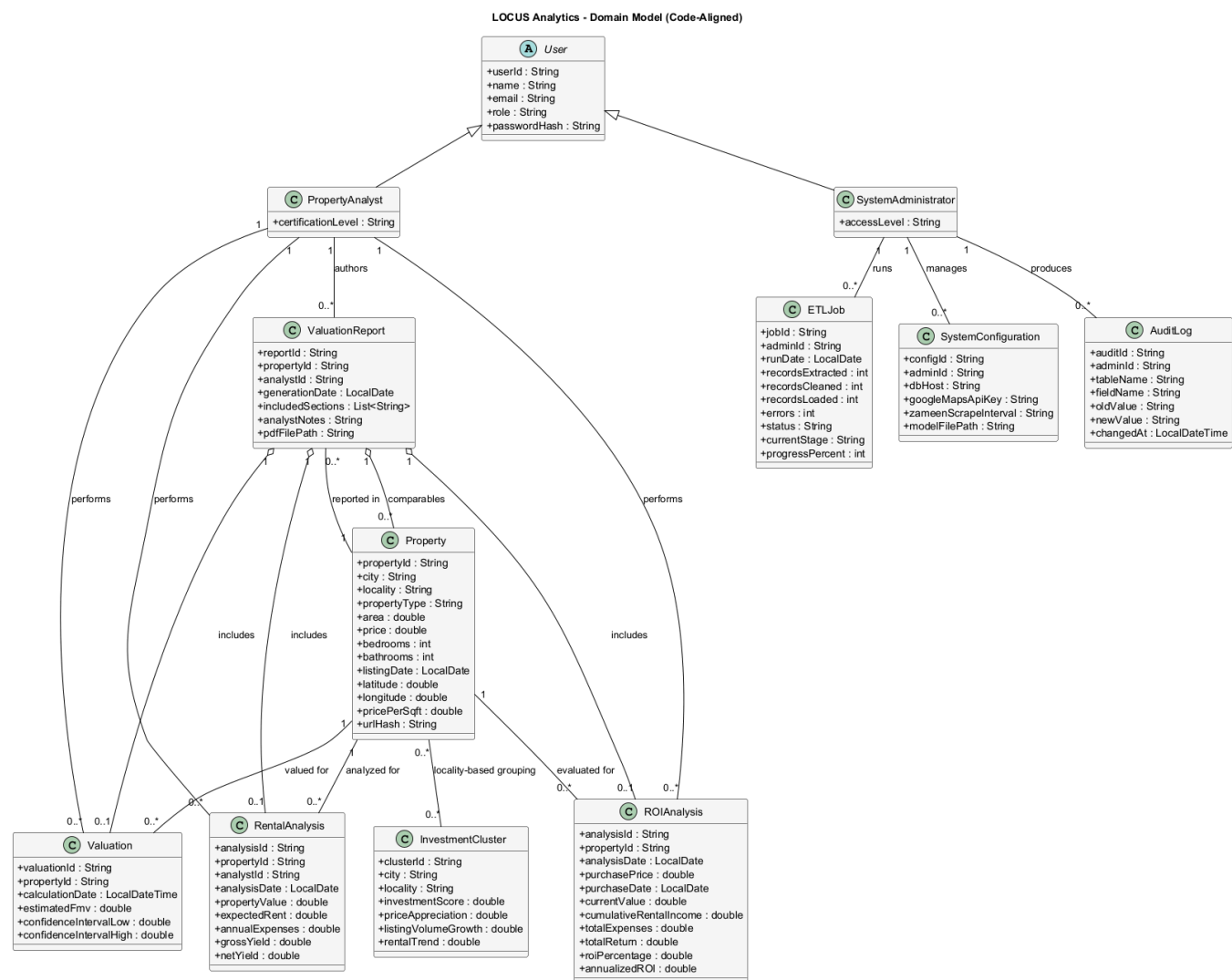


Figure 6.1 — LOCUS Analytics UML Domain Model.

6.1 Core Entities

Entity	Purpose
User / PropertyAnalyst / SystemAdministrator	Authentication and role-based capability gating.
Property	Listing record — location, area, beds, baths, price, geo coordinates.
Valuation	FMV estimate with lower / upper confidence bounds for a property.
RentalAnalysis	Annual rent, expenses, gross / net yield for an analyst-owned record.
ROIAnalysis	Purchase → current-value return, including annualised ROI.
ValuationReport	Composite PDF artifact tying selected sections together.
InvestmentCluster	Locality-level investment scoring based on appreciation + volume + rental trend.
ETLJob / SystemConfiguration	Admin-managed pipeline runs and global settings.
AuditLog	Trigger-driven trail of configuration mutations.

6.2 Relationships at a Glance

- PropertyAnalyst performs Valuation, initiates RentalAnalysis & ROIAnalysis, and generates ValuationReport.
- SystemAdministrator runs ETLJob and manages SystemConfiguration.
- Property is the central aggregate — every analytics record references it.
- InvestmentCluster belongs to one or many properties through locality matching.

7. Database Schema

The schema (`schema.sql`) defines 10 tables, a shared `update_timestamp()` trigger function, and an audit trigger that captures every mutation to `system_configuration` . Composite indexes are tuned to the access patterns of UC-4 (Search) and UC-6 (Price Trends).

7.1 Table Inventory

#	Table	PK	Notes
1	<code>app_user</code>	<code>user_id</code> (VARCHAR)	BCrypt <code>password_hash</code> ; role ∈
2	<code>property</code>	<code>property_id</code>	Generated <code>price_per_sqft</code> ; 7 indexes; unique <code>url_hash</code>
3	<code>valuation</code>	<code>valuation_id</code>	FMV + confidence interval; cascade on property
4	<code>rental_analysis</code>	<code>analysis_id</code>	<code>gross_yield</code> , <code>net_yield</code> ; <code>analyst_id</code> ref
5	<code>roi_analysis</code>	<code>analysis_id</code>	Purchase → current value, <code>annualized_roi</code>
6	<code>valuation_report</code>	<code>report_id</code>	PDF path + <code>included_sections</code> + <code>analyst_notes</code>
7	<code>investment_cluster</code>	<code>cluster_id</code>	Locality-level investment score
8	<code>etl_job</code>	<code>job_id</code>	<code>status</code> , <code>current_stage</code> , <code>progress_percent</code> (live UI)
9	<code>system_configuration</code>	<code>config_id</code>	DB host, GMaps key, scrape interval, model path
10	<code>audit_log</code>	<code>audit_id</code>	Triggered on <code>system_configuration</code> UPDATE

7.2 Index Strategy

- `idx_property_city_locality_type` : backbone for FMV comparables and Search filters.
- `idx_property_city_date` : powers Price Trend monthly aggregation per city.
- `idx_property_city_locality` : supports locality-scoped trend lookups with city fallback.
- `idx_valuation_property` / `idx_rental_property` / `idx_roi_property` : O(1) join from analytics to property.

7.3 Triggers

- `trg_*_BEFORE_UPDATE` on every mutable table — stamps `updated_at` .
- `trg_audit_config` — captures old/new JSON of `system_configuration` changes for compliance.

7.4 Hand-drawn Schema Reference

An informal hand-drawn ERD was used during the design phase to capture the relationships before formalising the DDL. The image below is the scanned reference (`Schema.pdf` in the repository root).

Legend: VARCHAR = VARCHAR(INDEX) PK → primary key
 DATE = DATE(INDEX) FK → foreign key

Date: / /

MTWTFSS

Property	
property-id	VARCHAR(PK)
city	VARCHAR(INDEX)
locality	VARCHAR(INDEX)
property-type	VARCHAR
area	FLOAT
price	FLOAT
bedrooms	INT
bathrooms	INT
listing-date	DATE(INDEX)
latitude	FLOAT
longitude	FLOAT
price-per-sqft	(derived: price/area)
url-hash	VARCHAR(UNIQUE)

Valuation	
valuation-id	VARCHAR(PK)
property-id	[Property.property-id]
estimation-time	FLOAT
confidence-lower-limit	Float
confidence-upper-limit	Float

USER	
userID	VARCHAR(PK)
name	VARCHAR
email	VARCHAR
role	VARCHAR
certification-level	VARCHAR
access-level	VARCHAR

Rental Analysis	
analysis-id	VARCHAR(PK)
property-id	VARCHAR(FK)
annual-rent	FLOAT
annual-expenses	FLOAT
gross-yield	FLOAT
net-yield	FLOAT
analysis-id	VARCHAR(FK)
annual-date	DATE

ETLJOB	
adminID	VARCHAR(FK)
jobID	VARCHAR(PK)
runDate	DATE
records-extracted	INT
records-changed	INT
records-loaded	INT
errors	INT
status	VARCHAR

ValuationReport	
reportID	VARCHAR(PK)
property-id	VARCHAR(FK)
analyst-id	VARCHAR(FK)
generationDate	DATE
includedSections	VARCHAR
analystNotes	VARCHAR
pdfFileData	VARCHAR

ROI Analysis	
analysis-id	VARCHAR(PK)
property-id	VARCHAR(FK)
analysisDate	DATE
purchasePrice	FLOAT
purchaseDate	DATE
cumulativeRentalIncome	FLOAT
totalExpenses	FLOAT
totalReturn	FLOAT
roi-percentage	FLOAT
annualizedROI	FLOAT

System Configuration	
configID	VARCHAR(PK)
adminID	VARCHAR(FK)
dbHost	VARCHAR
googleMapsApiKey	VARCHAR
ZenecaScriptInterval	VARCHAR
modelFilePath	VARCHAR

Investment Cluster	
clusterID	VARCHAR(PK)
city	VARCHAR
locality	VARCHAR
investmentScore	FLOAT
priceAppreciation	FLOAT
listingScheduleFromThu	FLOAT

BT

Figure 7.1 — Hand-drawn ERD (rendered from Schema.pdf).

8. Use Case Specifications (UCS)

LOCUS Analytics implements twelve use cases, each owned by a dedicated service and exposed through a single FXML screen. The first nine are available to both analysts and admins; UC-10 through UC-12 are admin-only and gated at sidebar render time.

#	Use Case	Screen	Primary Service
UC-1	Estimate Fair Market Value	FMV Estimate	ValuationService
UC-2	Calculate Rental Yield	Rental Yield	RentalYieldService
UC-3	Calculate ROI	ROI Calculator	ROIService
UC-4	Search Properties	Search	SearchService
UC-5	Compare Properties	Compare	CompareService
UC-6	View Price Trends	Price Trends	PriceTrendService
UC-7	Property Heatmap	Heatmap	HeatmapService
UC-8	Generate Valuation Report (PDF)	Reports	ValuationReportService + ReportPdfService
UC-9	Identify Investment Clusters	Clusters	InvestmentClusterService
UC-10	Run ETL Pipeline	ETL Dashboard (Admin)	ETLService
UC-11	Manage Listings	Listings (Admin)	ListingManagementService
UC-12	System Configuration	Config (Admin)	ConfigurationService

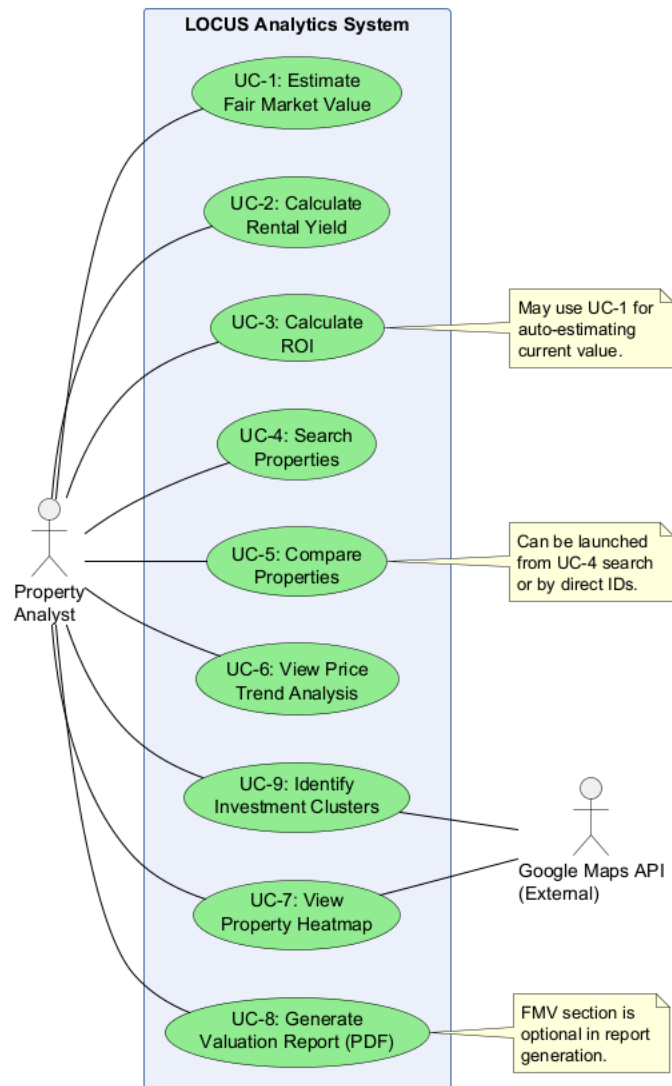
8.1 Use Case Diagrams

The diagrams below depict the primary actors (Property Analyst, System Administrator), the supporting external actors (Google Maps API, Zameen.com), and optional dependencies that match implementation behavior. The Administrator inherits all Analyst capabilities through actor generalisation.

UCS - Analyst Use Case Diagram

Covers UC-1 through UC-9, available to both analysts and admins. Notes capture optional behavior: UC-3 may use UC-1 for auto-FMV, UC-5 can be launched from UC-4 search or direct property IDs, and UC-8 may include FMV as an optional report section. UC-7 and UC-9 keep plain associations to the Google Maps API.

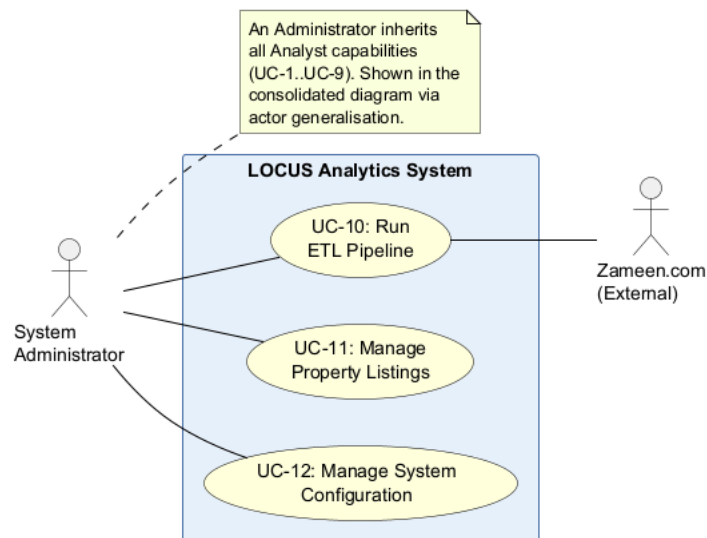
UCS - Analyst Use Case Diagram



UCS - Administrator Use Case Diagram

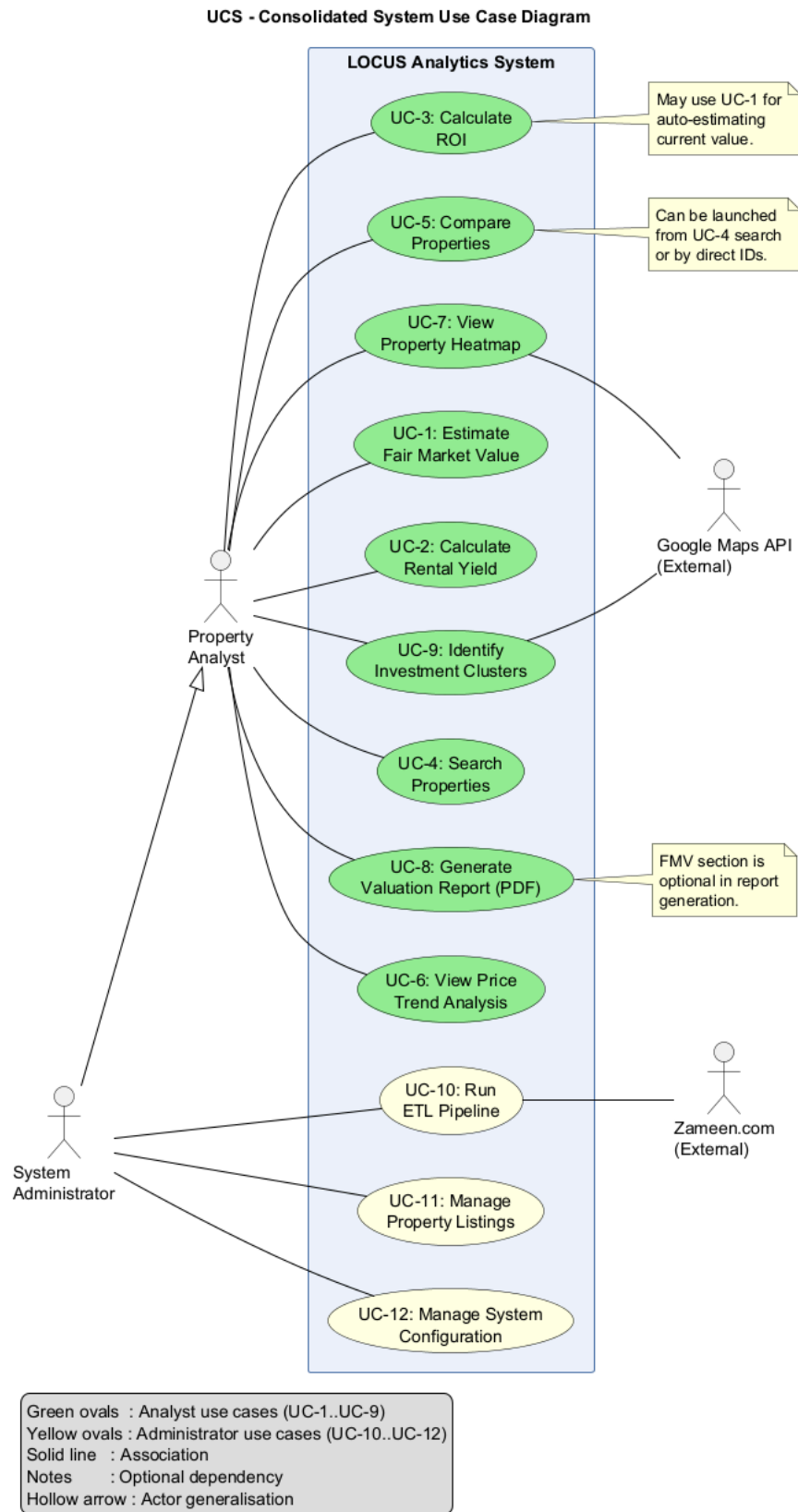
Covers the admin-only UC-10 through UC-12. UC-10 (Run ETL Pipeline) has a plain association to the external Zameen.com data source.

UCS - Administrator Use Case Diagram



UCS - Consolidated System Use Case Diagram

Combined system-level diagram. Both actors and all 12 use cases. The actor generalisation arrow `System Administrator` $\rightarrow$ `Property Analyst` indicates that admins can also perform any analyst use case.



9. System Sequence Diagrams (SSDs)

System Sequence Diagrams capture the conversation between the actor and the system as a black box—here the system participant is labelled `:LOCUS` (equivalent to `:System` in textbook SSDs). Messages match the fully dressed use cases and the implementation layers (`Service` / `UI`) only appear in \$10 design-level SDs.

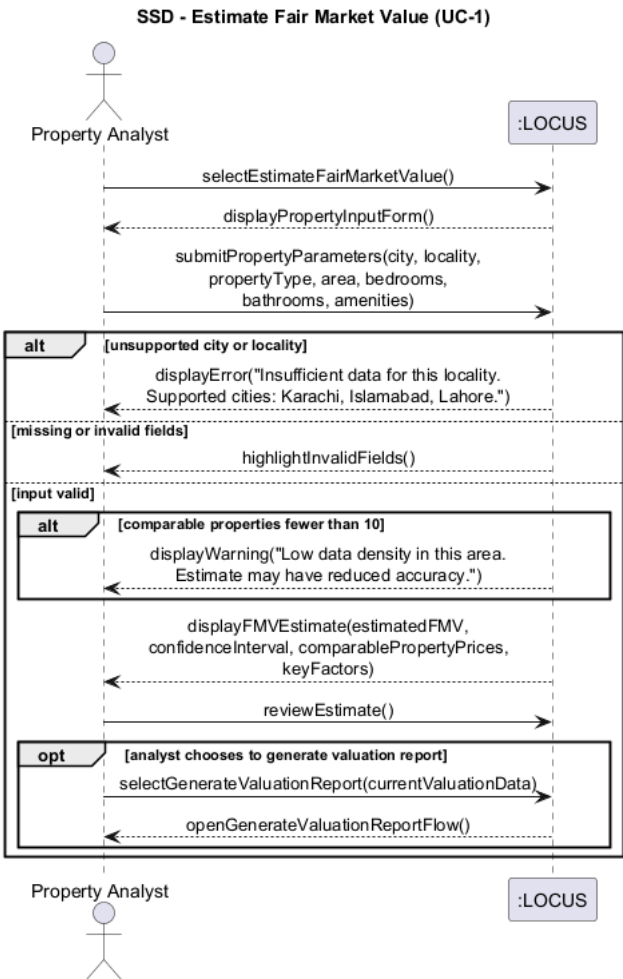
Editable sources: `SSD_UC-1.puml` ... `SSD_UC-12.puml` in `docs/sequence-diagrams/`.

Audit notes

All 12 SSDs are aligned to the codebase as two-lifeline system views (Actor + `:LOCUS`) with key extensions modelled as `opt` / `alt` fragments. The first six analyst flows cover FMV validation and low-comparable warnings (UC-1), rental-yield validation with city-level benchmark messaging (UC-2), ROI auto-FMV and short-holding warning (UC-3), search empty-results / pagination / drill-down to UC-5 (UC-4), compare bounds and FMV-on-demand handoff to UC-1 (UC-5), and trend locality fallback plus overlays/zoom (UC-6). The remaining six cover heatmap success/fallback (UC-7), report-generation validation and open/save/print outcomes (UC-8), cluster ranking with optional locality handoff to Search (UC-9), ETL start/poll/fail/success phases with completion summary (UC-10), Add/Update/Delete listing management with validation and DB-failure branches (UC-11), and configuration no-change/validation/save-failure/success outcomes (UC-12). SSD sources were updated to match current implementation behavior.

SSD — UC-1: Estimate Fair Market Value

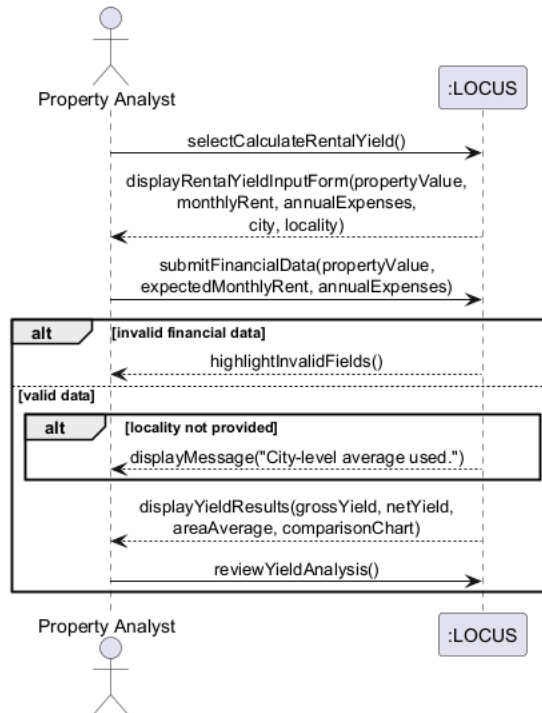
Actor: Property Analyst. Extensions: unsupported locality (3a), invalid fields (4a), low comparables warning (5a), optional valuation report (9a).



SSD — UC-2: Calculate Rental Yield

Extensions: invalid financial data (4a), city-level average messaging when locality is not provided (6a).

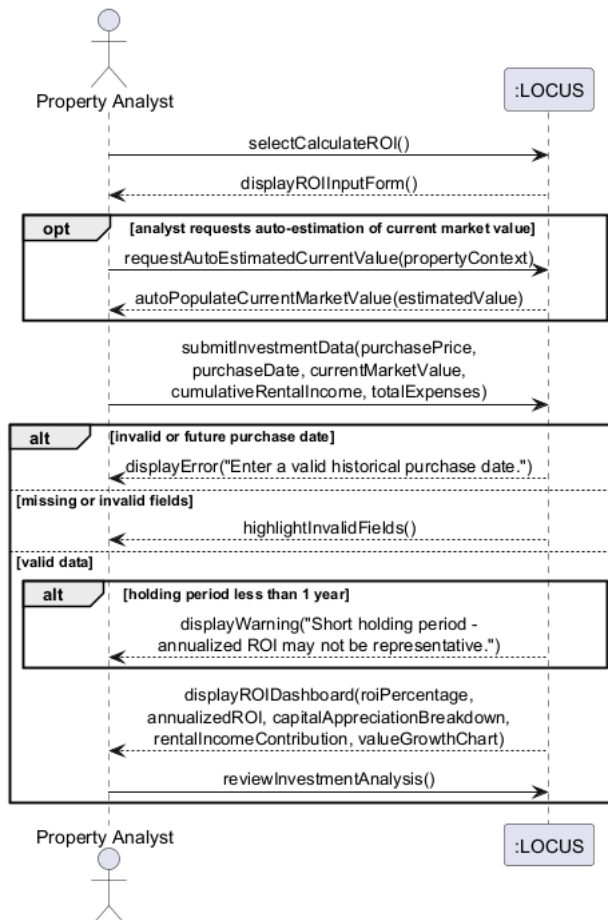
SSD - Calculate Rental Yield (UC-2)



SSD — UC-3: Calculate ROI

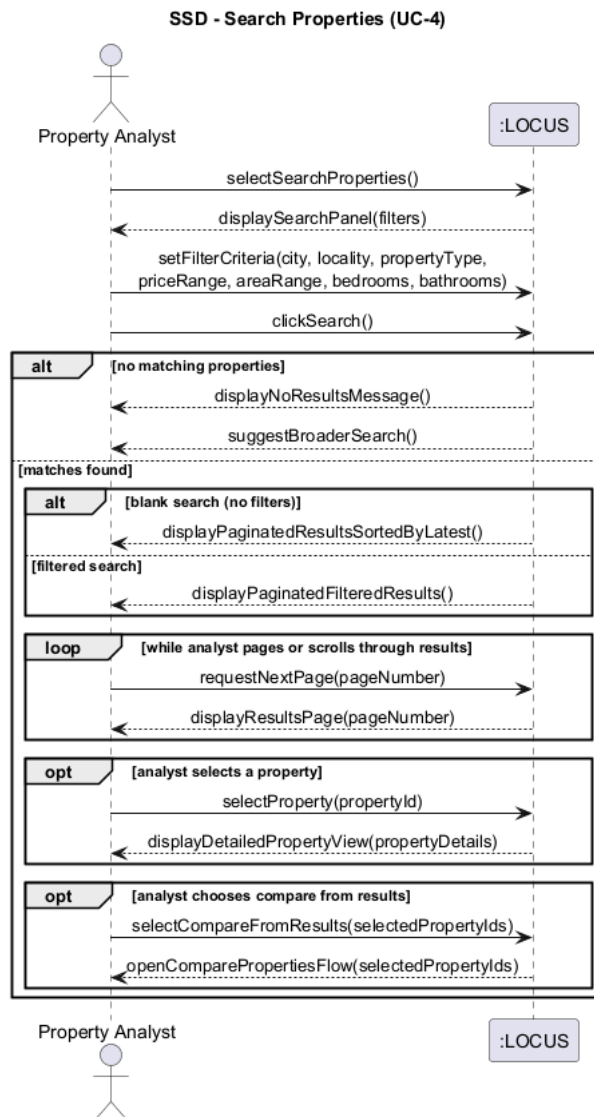
Extensions: auto-estimated current value via FMV path (2a), invalid / future purchase date (4a), short holding-period warning (5a).

SSD - Calculate ROI (UC-3)



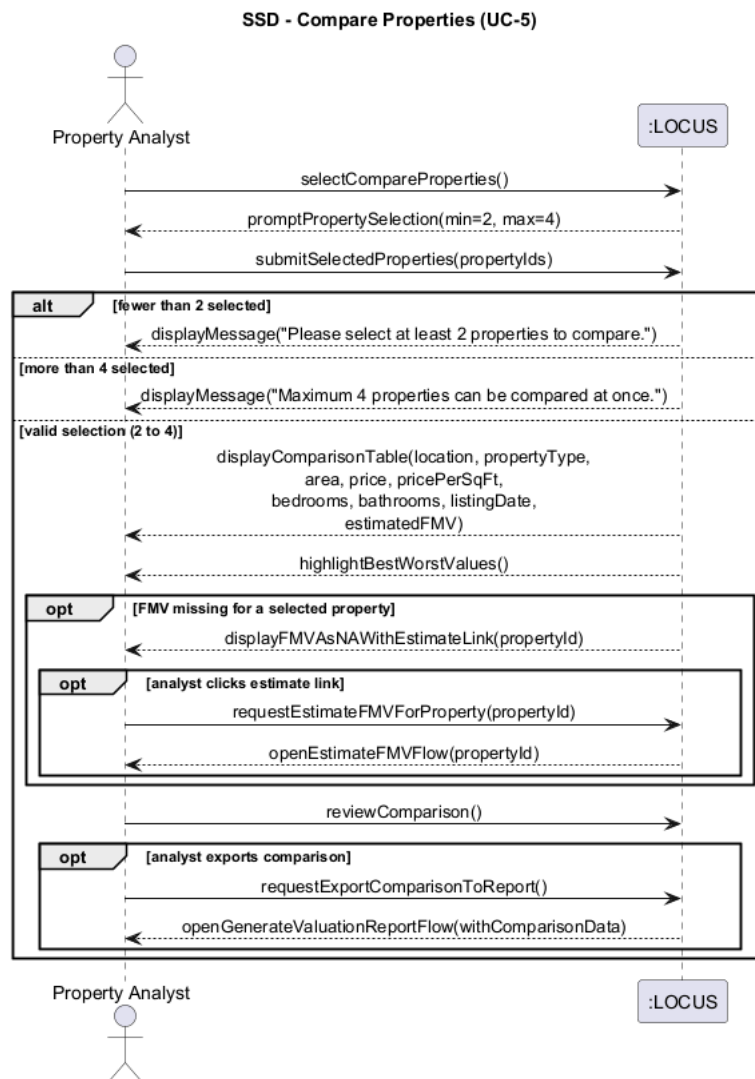
SSD — UC-4: Search Properties

Extensions: blank search vs filtered (3a / main flow), no matches (6a), pagination (main success), property detail (8a), invoke compare (8b).



SSD — UC-5: Compare Properties

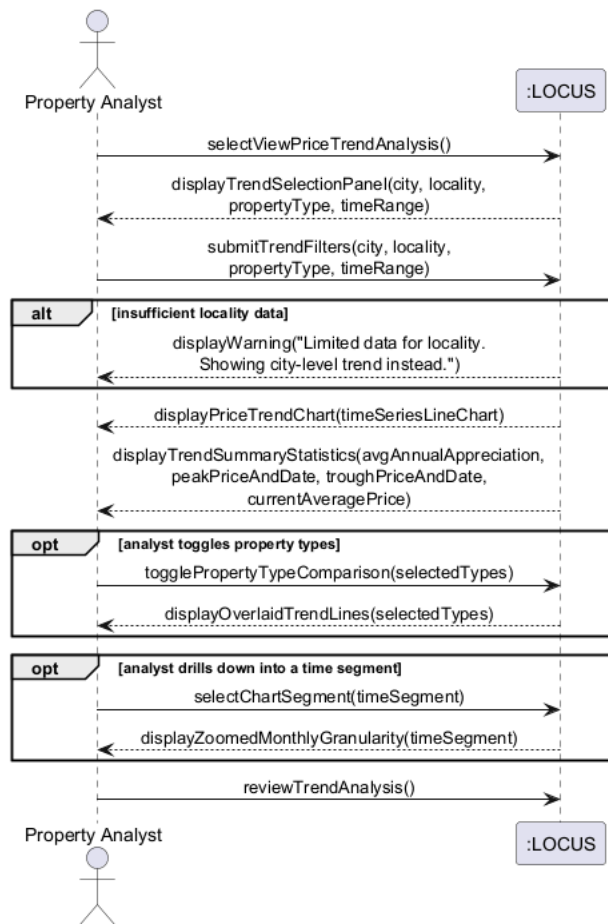
Extensions: fewer than 2 / more than 4 properties (3a / 3b), missing FMV with estimate link (5a), export toward valuation report (7a).



SSD — UC-6: View Price Trend Analysis

Extensions: locality fallback when sparse data (4a), overlay property types (6a), zoom / monthly drill-down (8a).

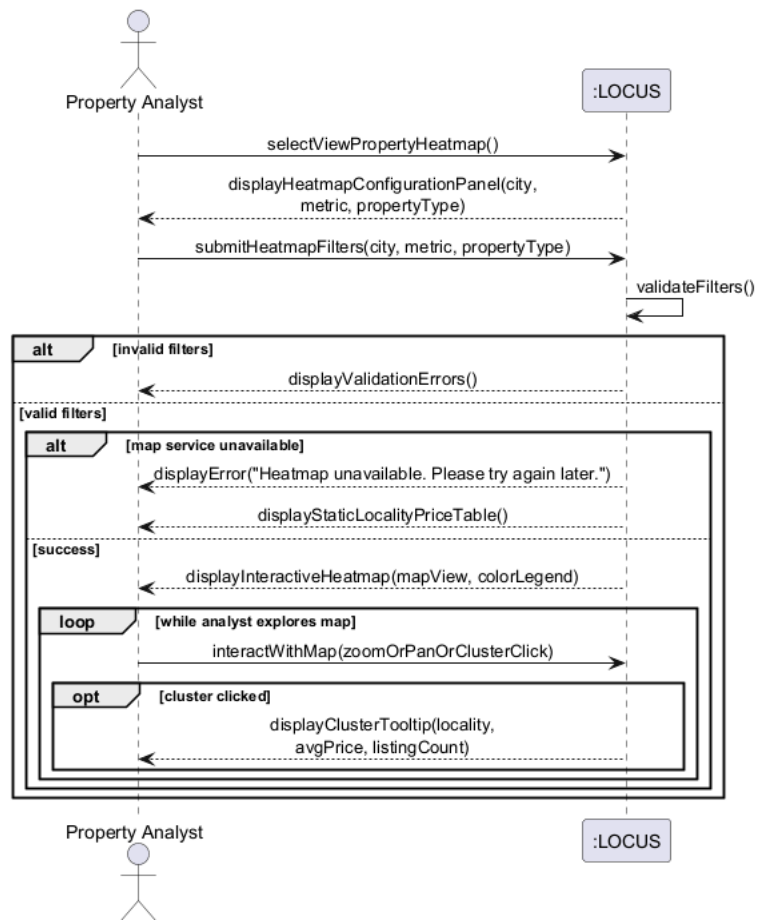
SSD - View Price Trend Analysis (UC-6)



SSD — UC-7: View Property Heatmap

Internal `validateFilters()` self-call. Extensions: invalid filters, map service unavailable (static fallback table), interactive map exploration with tooltip on cluster click.

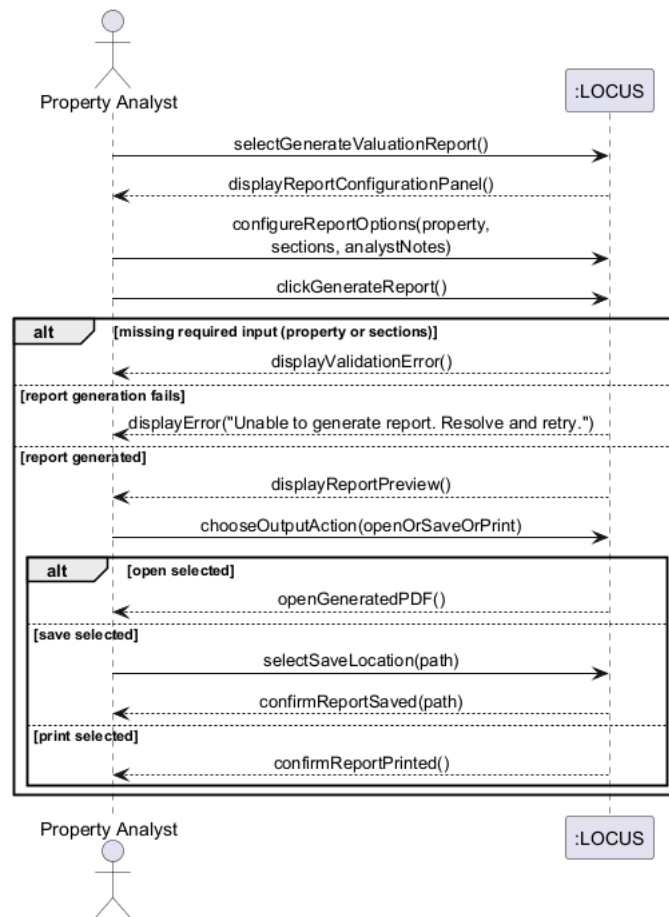
SSD - View Property Heatmap (UC-7)



SSD — UC-8: Generate Valuation Report (PDF)

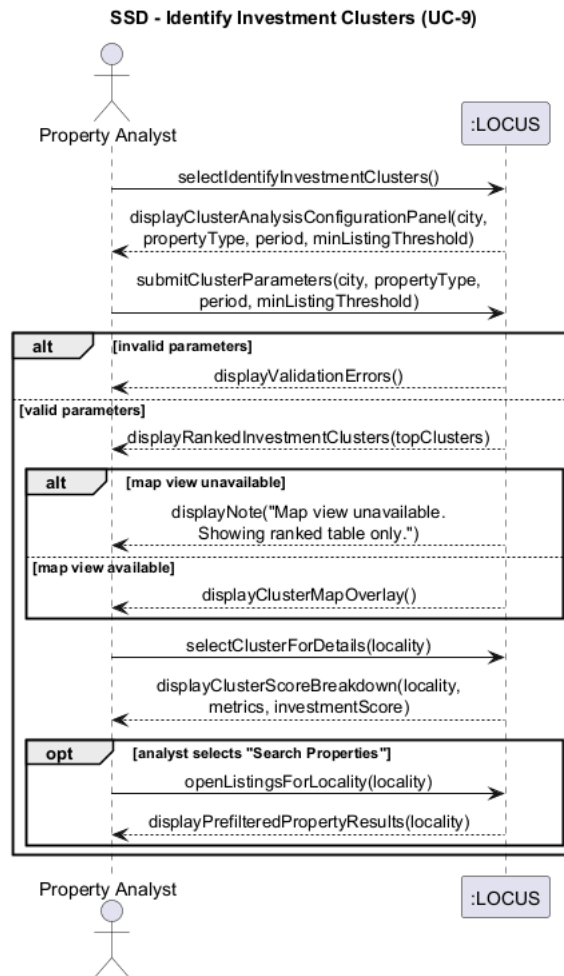
Extensions: missing required inputs (property/sections), report-generation failure, and output actions (open/save/print).

SSD - Generate Valuation Report (UC-8)



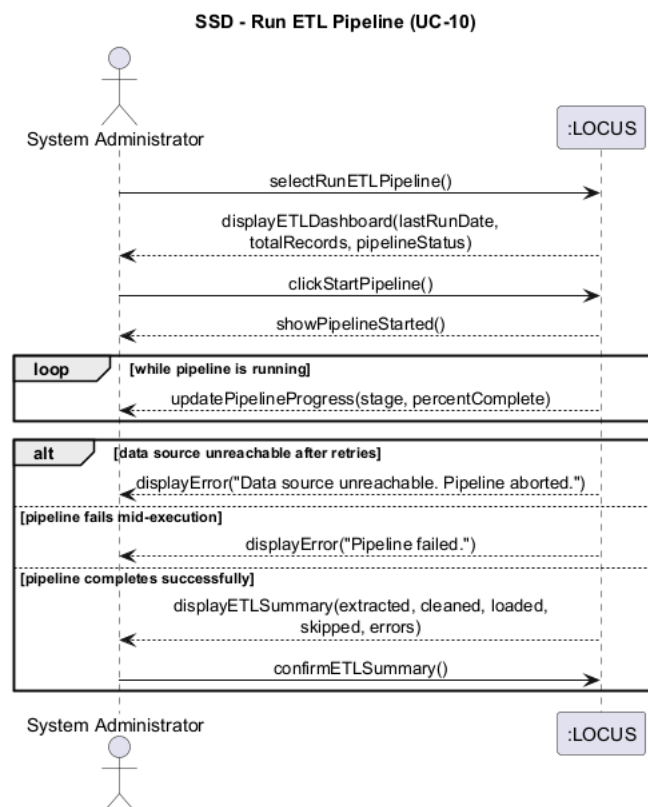
SSD — UC-9: Identify Investment Clusters

Extensions: invalid parameters, map view unavailable (ranked table only), drill-down into a cluster, optional handoff to "Search Properties" with locality prefilter.



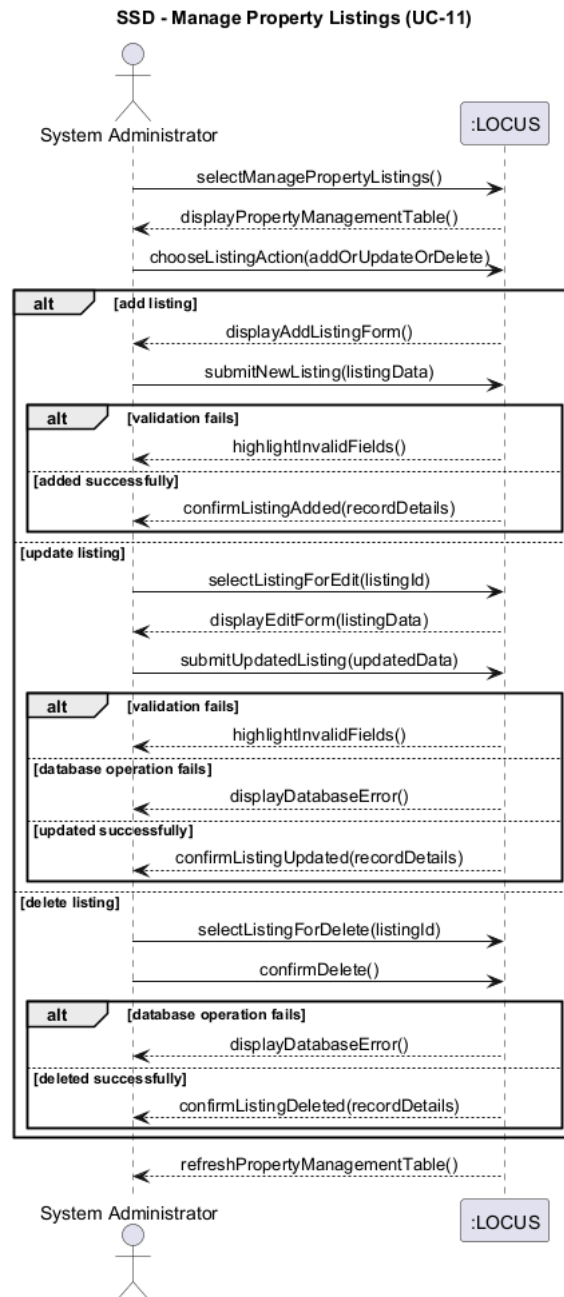
SSD — UC-10: Run ETL Pipeline

Pipeline progress shown as a `loop`, terminal states modelled as `alt` (source unreachable, mid-run failure, success with summary confirmation).



SSD — UC-11: Manage Property Listings

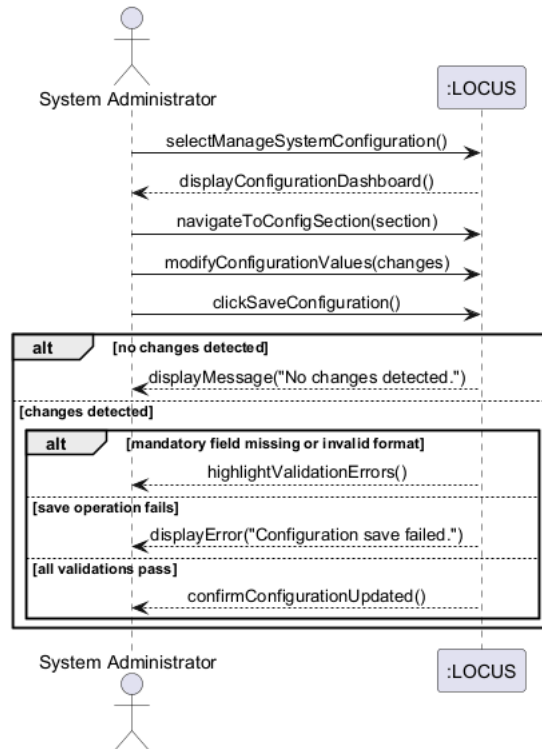
Three primary `alt` branches (Add / Update / Delete). Add and update include validation/error paths; delete includes confirmation and DB-failure branch. All paths converge on `refreshPropertyManagementTable()`.



SSD — UC-12: Manage System Configuration

Validation cascade: `[no changes]` short-circuits; `[changes]` covers mandatory-field checks, save-operation failure, and successful update confirmation.

SSD - Manage System Configuration (UC-12)

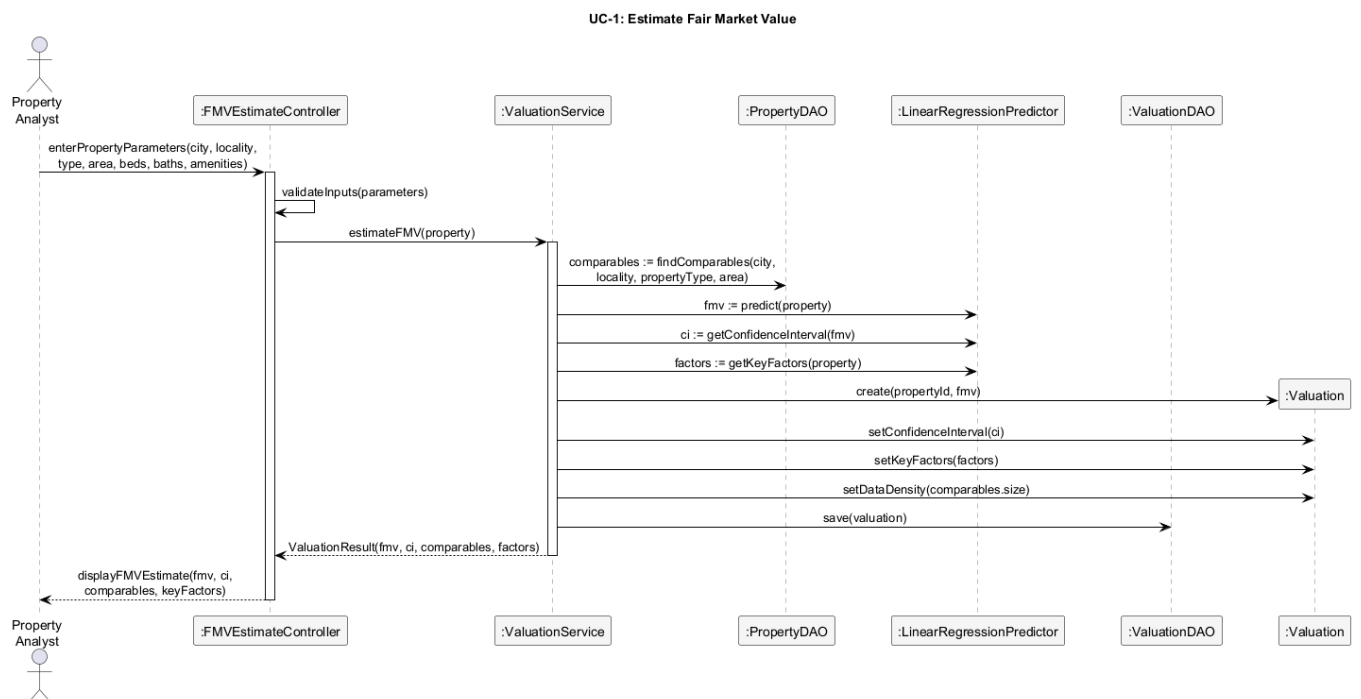


10. Sequence Diagrams (SDs)

Sequence Diagrams open the system box and depict the inter-object messages across UI Controller $\leftrightarrow$ Service $\leftrightarrow$ DAO $\leftrightarrow$ Database (and, for UC-1, the `LinearRegressionPredictor`). Participants below are the actual classes from the codebase: controllers under `com.locus.ui.controller.screen`, service interfaces under `com.locus.service`, DAOs under `com.locus.dao`, and the predictor under `com.locus.ml`.

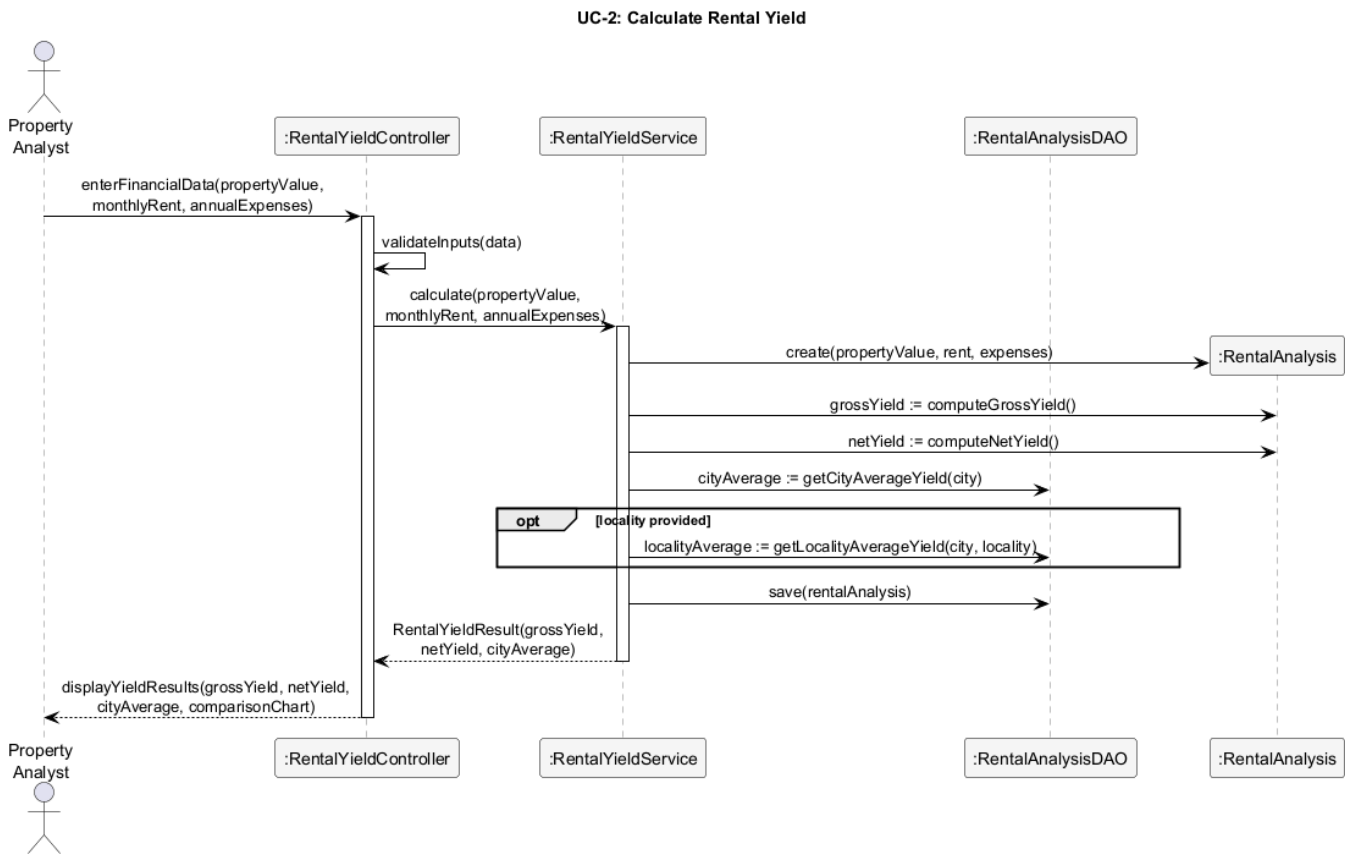
SD — UC-1: Estimate Fair Market Value

`FMVEstimateController` $\rightarrow$ `ValuationService` $\rightarrow$ `LinearRegressionPredictor` + `PropertyDAO` $\rightarrow$ `ValuationDAO`.



SD — UC-2: Calculate Rental Yield

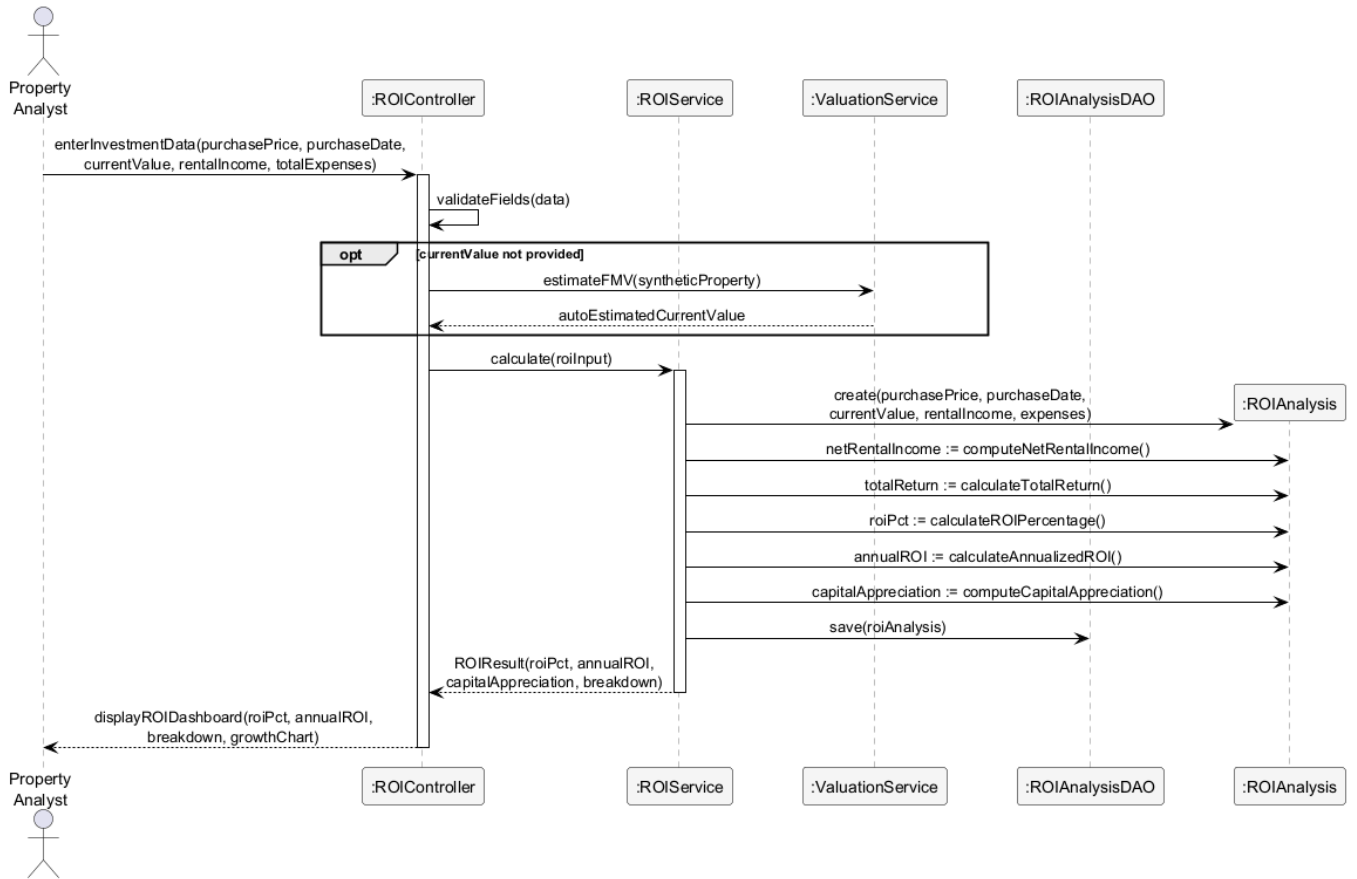
RentalYieldController → RentalYieldService → RentalAnalysisDAO (calculation + benchmark lookup + persistence).



SD — UC-3: Calculate ROI

ROIController → ROIService → ROIAnalysisDAO, with optional ValuationService call from controller for auto-estimated current value.

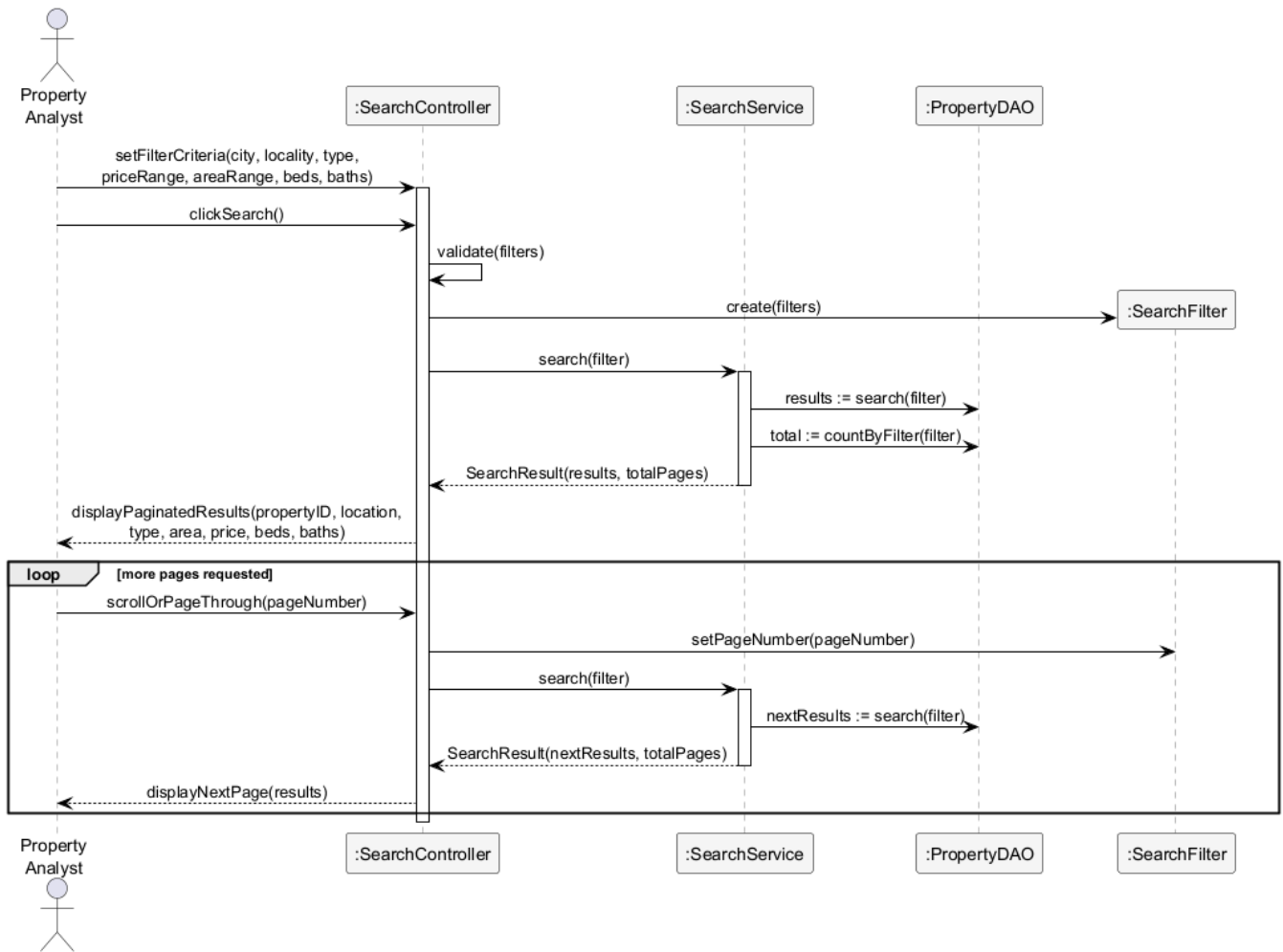
UC-3: Calculate ROI (Return on Investment)



SD — UC-4: Search Properties

SearchController → SearchService → PropertyDAO using SearchFilter / PagedResult pagination flow.

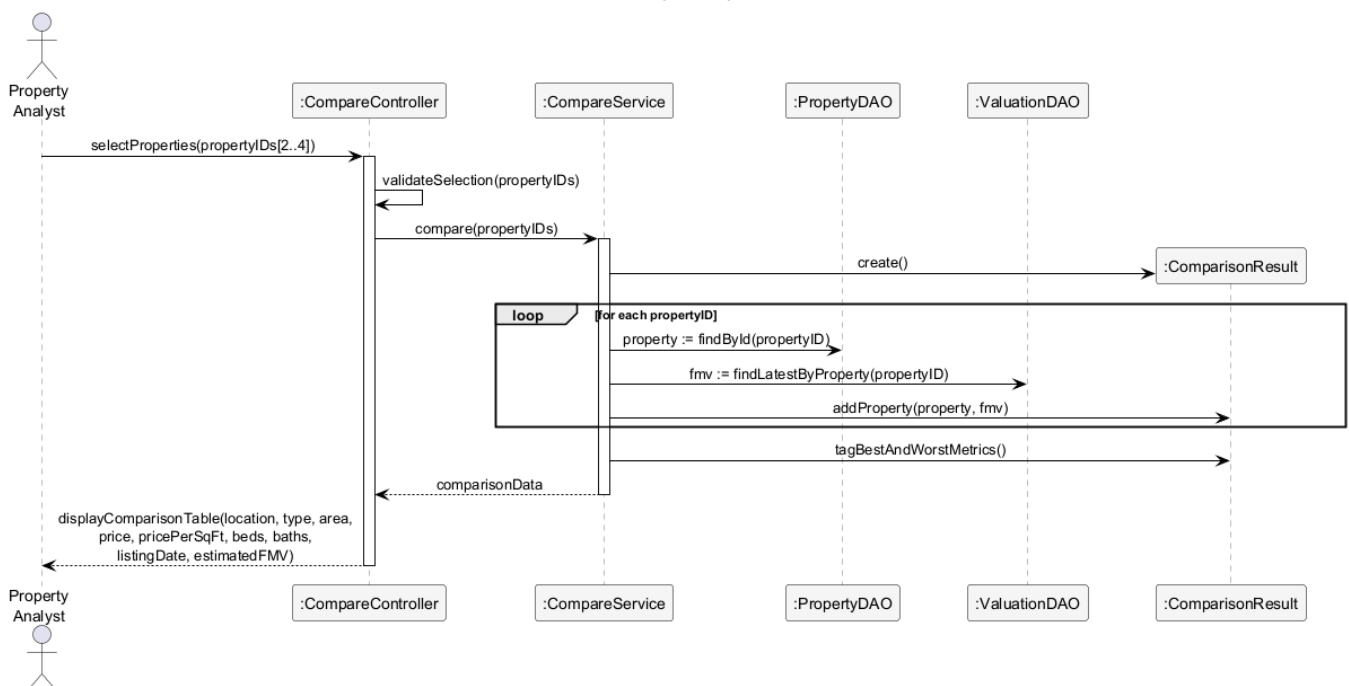
UC-4: Search Properties



SD — UC-5: Compare Properties

CompareController → CompareService → PropertyDAO + ValuationDAO producing ranked ComparisonResult flags.

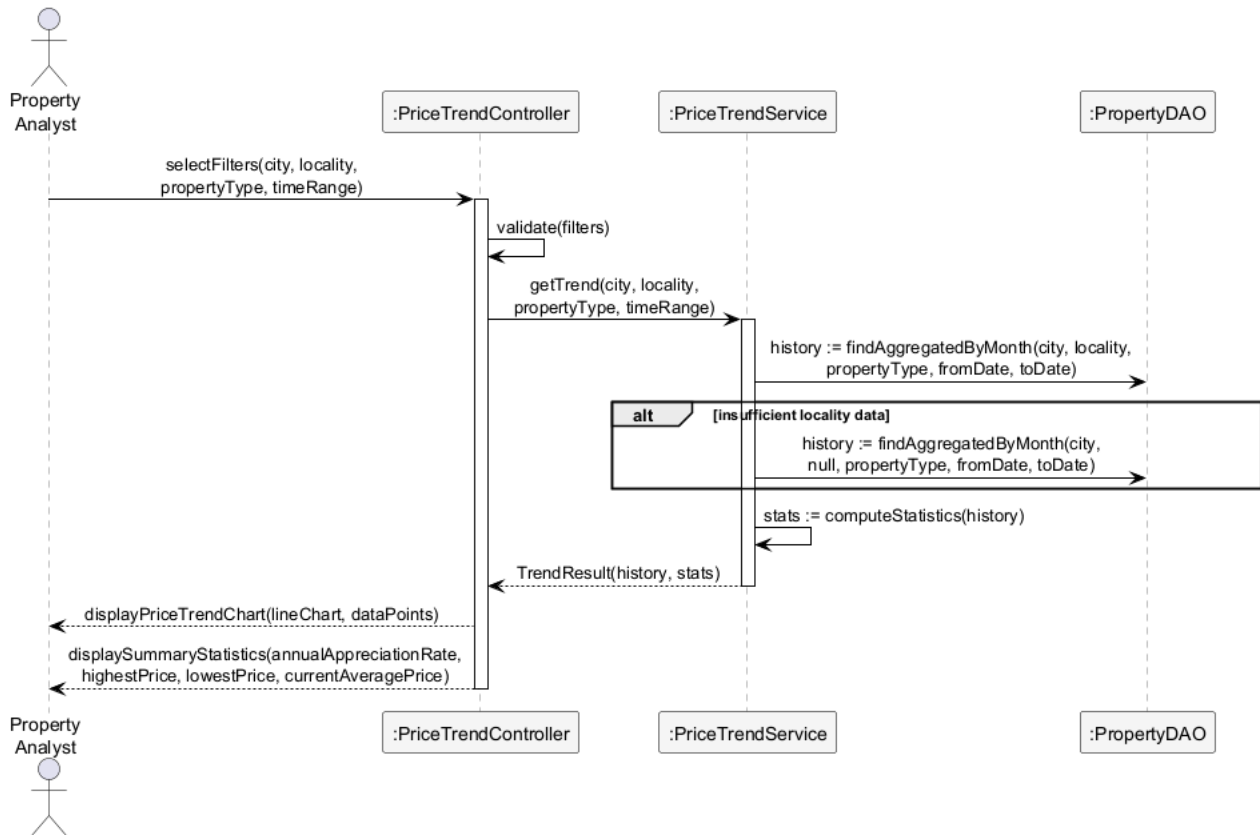
UC-5: Compare Properties



SD — UC-6: View Price Trend Analysis

PriceTrendController → PriceTrendService → PropertyDAO with locality-to-city fallback and computed statistics.

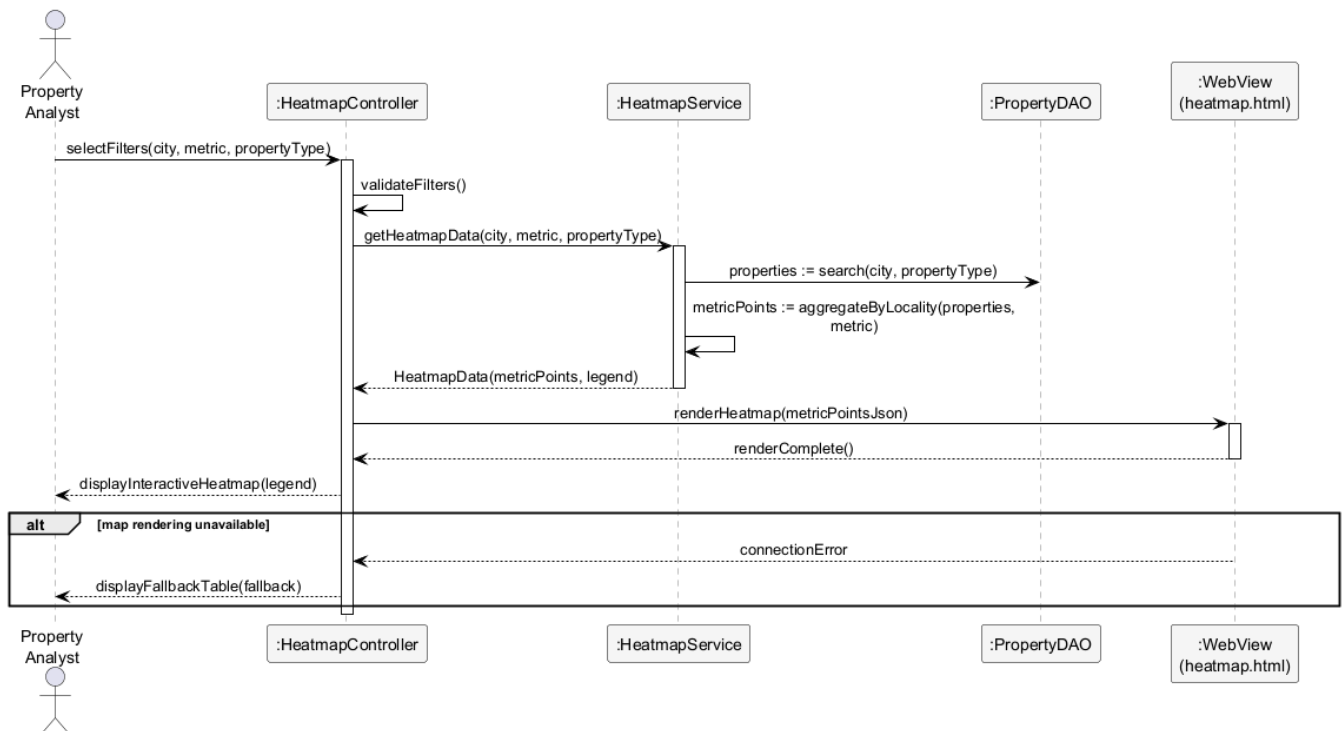
UC-6: View Price Trend Analysis



SD — UC-7: View Property Heatmap

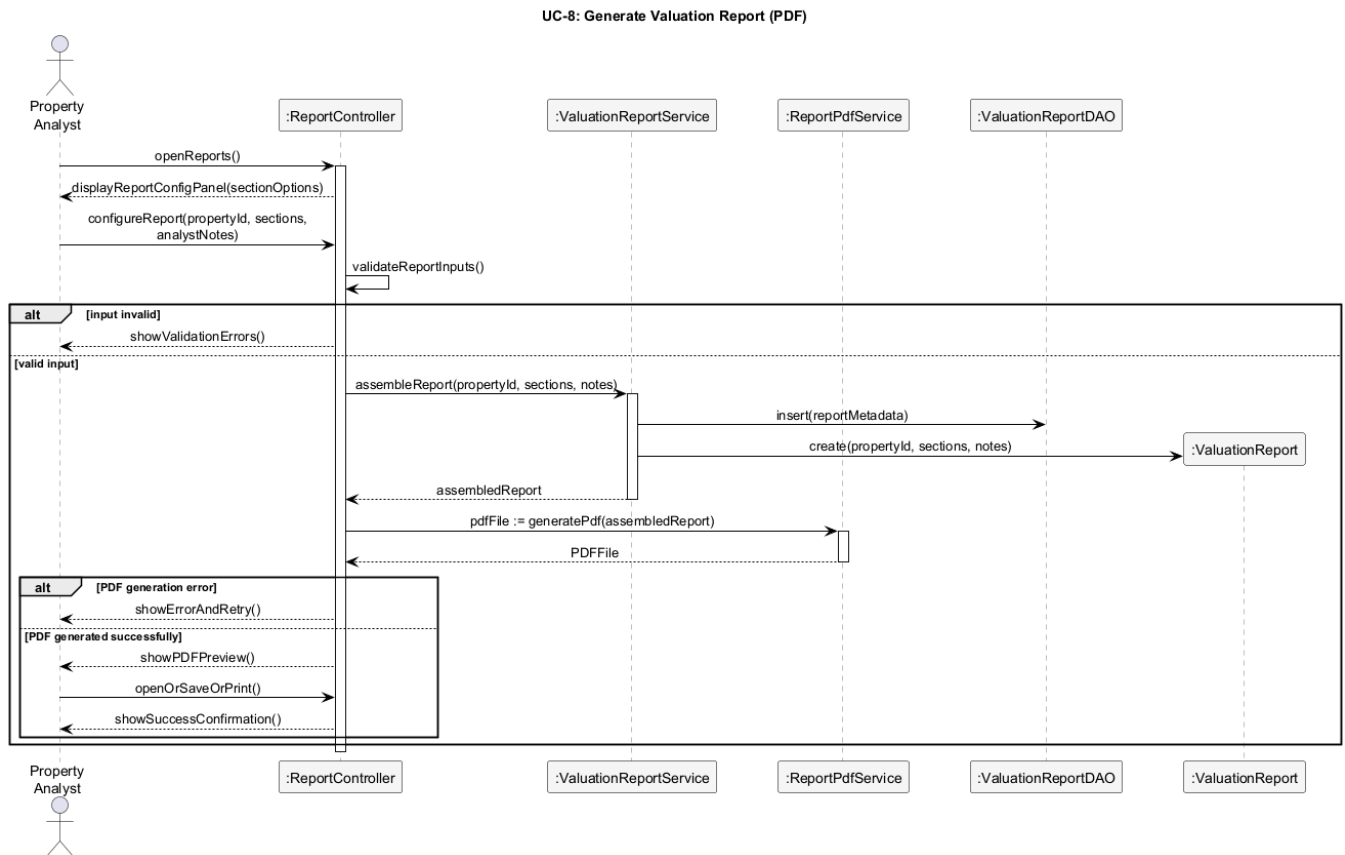
HeatmapController → HeatmapService → PropertyDAO (and rental benchmark data inside service); embedded WebView renders heatmap.html.

UC-7: View Property Heatmap



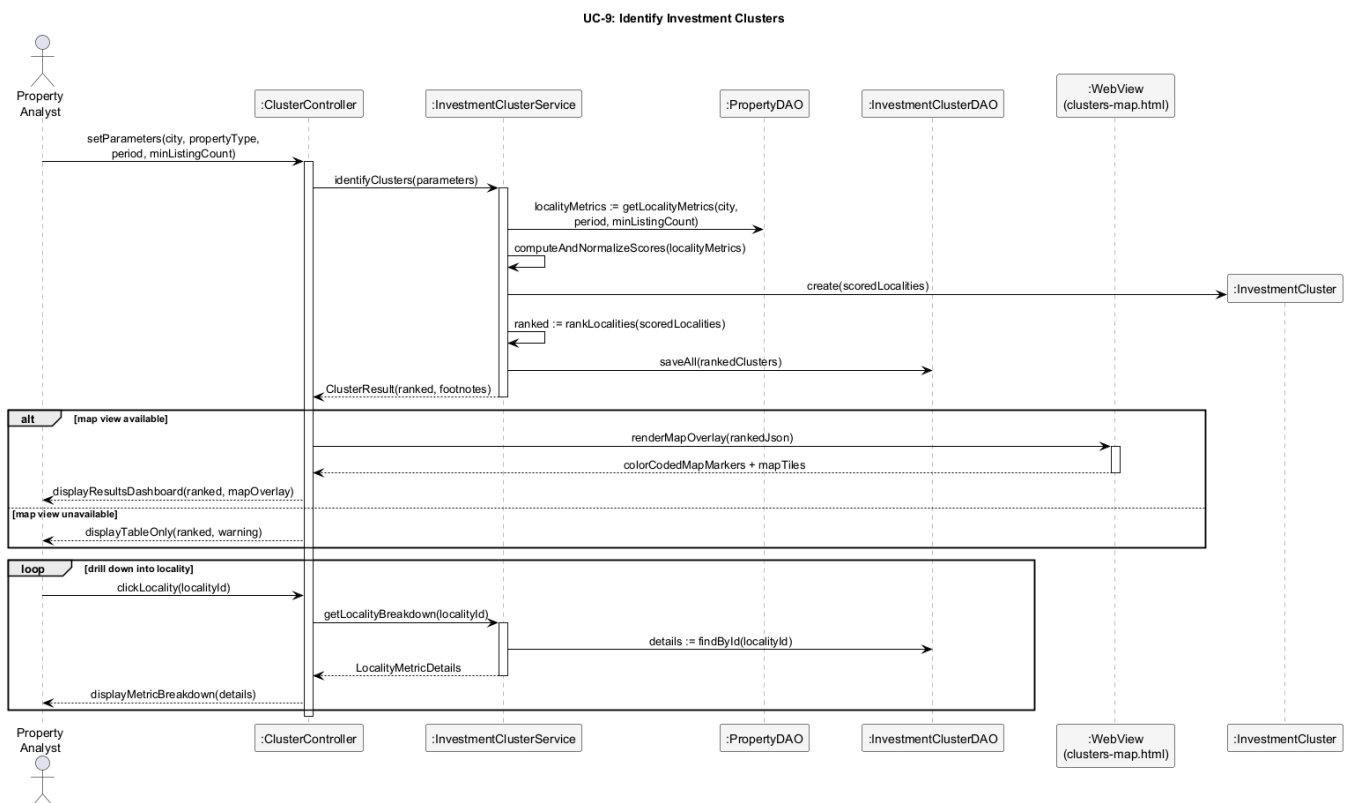
SD — UC-8: Generate Valuation Report (PDF)

ReportController → ValuationReportService (assemble data + persist metadata) → ReportPdfService (PDFBox) with controller-side open/save/print actions.



SD — UC-9: Identify Investment Clusters

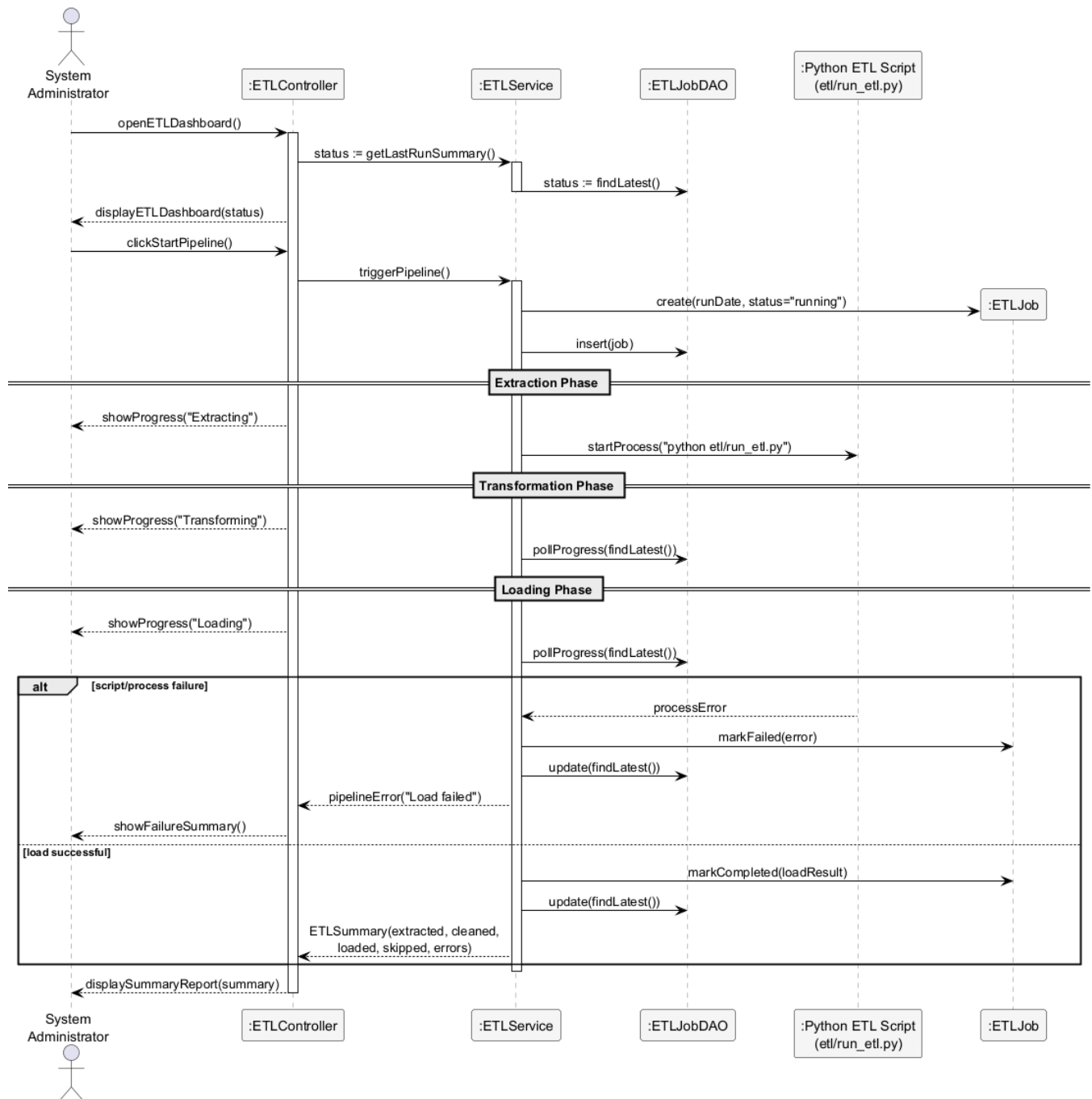
ClusterController → InvestmentClusterService → PropertyDAO + InvestmentClusterDAO (+ rental benchmark source in service); WebView renders clusters-map.html.



SD — UC-10: Run ETL Pipeline

ETLController → ETLService → Python ETL process (etl/run_etl.py) with status polling through ETLJobDAO .

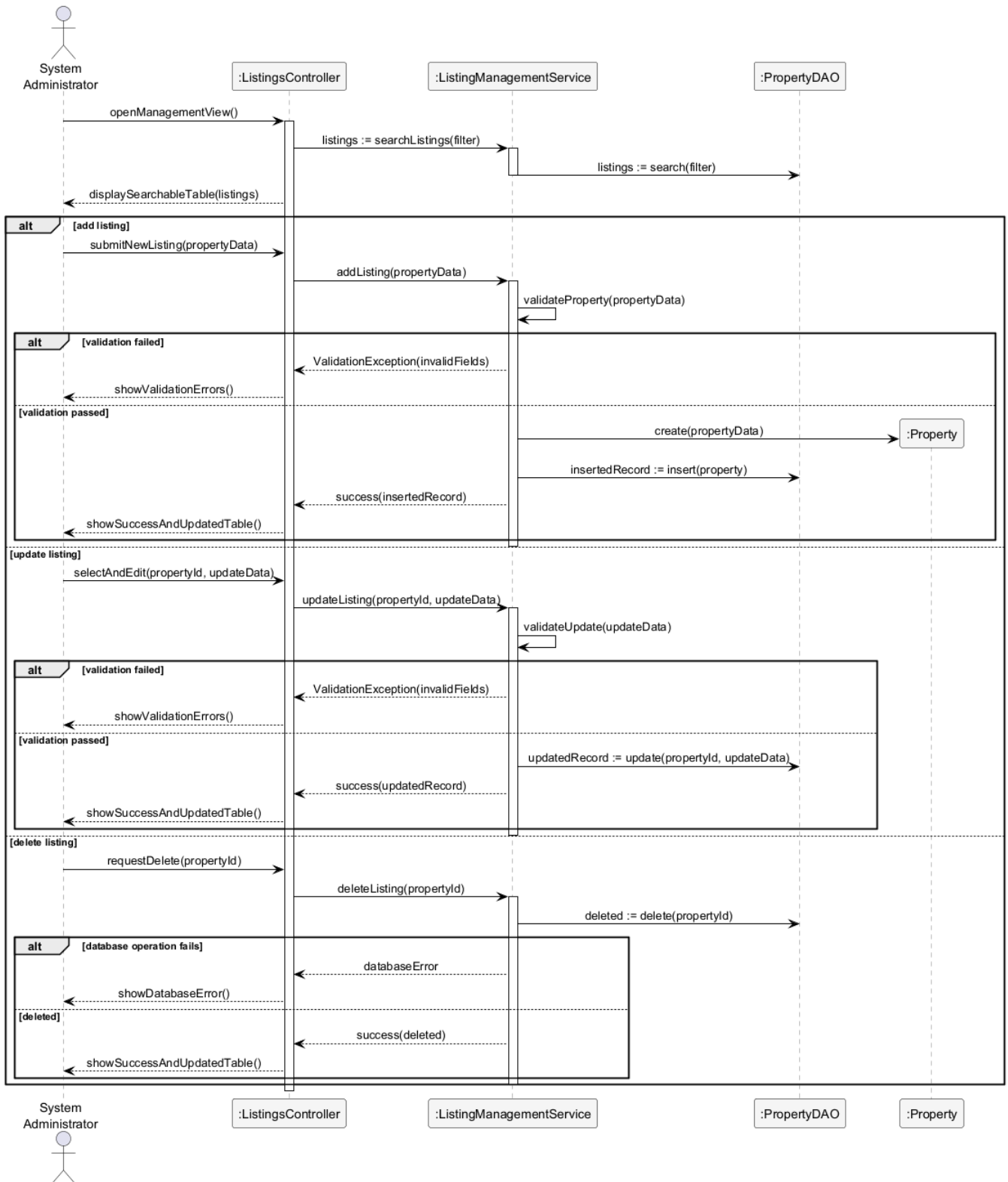
UC-10: Run ETL Pipeline



SD — UC-11: Manage Property Listings

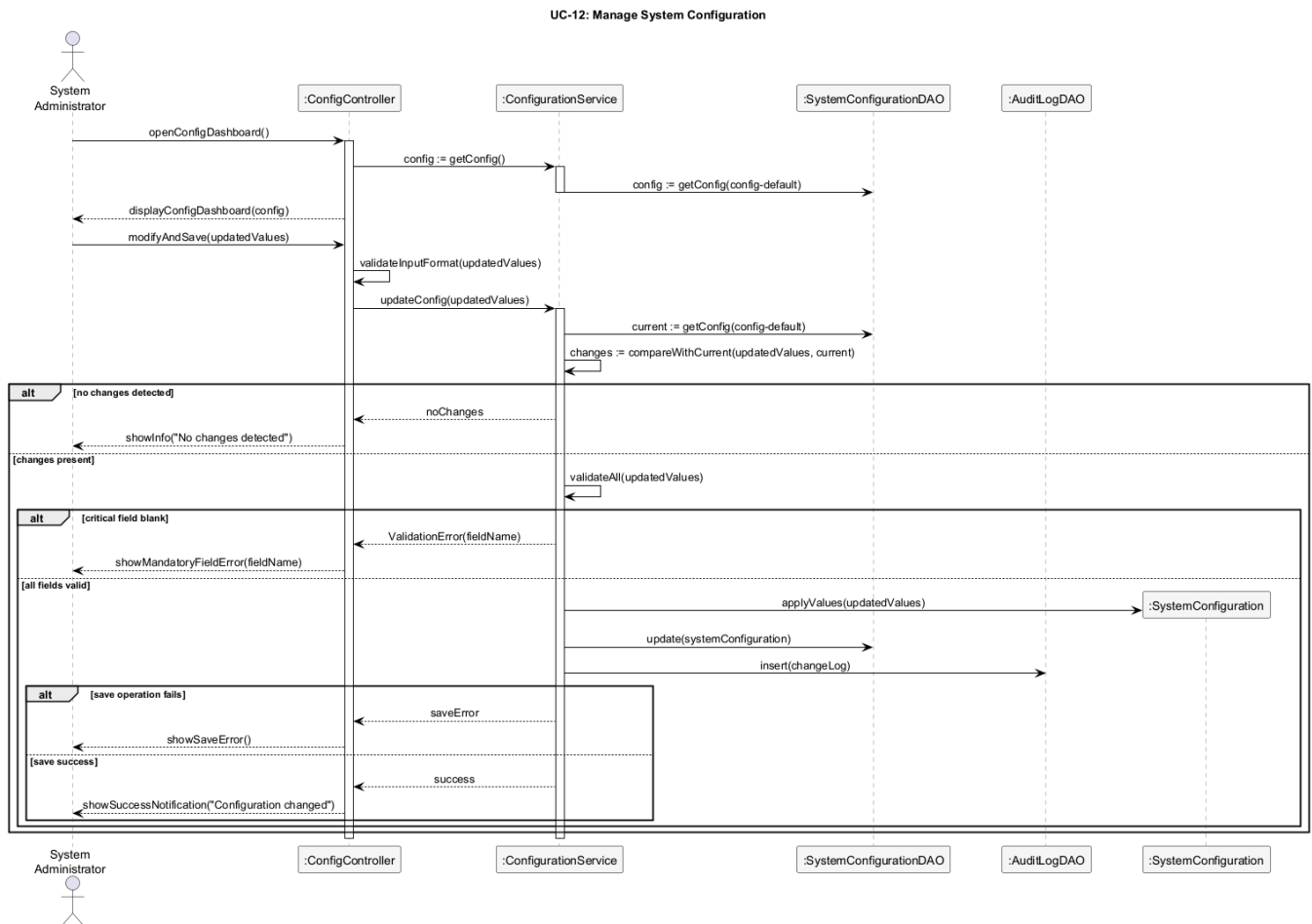
ListingsController → ListingManagementService → PropertyDAO (validated CRUD + paged search).

UC-11: Manage Property Listings



SD — UC-12: Manage System Configuration

ConfigController → ConfigurationService → SystemConfigurationDAO + AuditLogDAO with format validation and save/error confirmation flow.



11. Design Class Diagram

The diagram below is the implementation-level class diagram for LOCUS Analytics, derived directly from the Java source tree under `locus-analytics/src/main/java/com/locus/`. It complements the conceptual Domain Model in §6 by adding the application layers — JavaFX FXML controllers, business-service interfaces, JDBC DAO interfaces, and the ML predictor — together with the concrete dependencies that wire them together.

11.1 Layering & Stereotypes

Stereotype	Layer	Java package
«Controller»	JavaFX FXML controller (UI)	<code>com.locus.ui.controller</code> and <code>com.locus.ui.controller.screen</code>
«Service»	Business service interface	<code>com.locus.service</code> (impls in <code>...service.impl</code>)
«DAO»	JDBC data-access interface	<code>com.locus.dao</code> (impls in <code>...dao.impl</code>)
«Predictor»	ML model wrapper (Ridge regression)	<code>com.locus.ml</code>
(entity)	Persistent domain class	<code>com.locus.model</code>

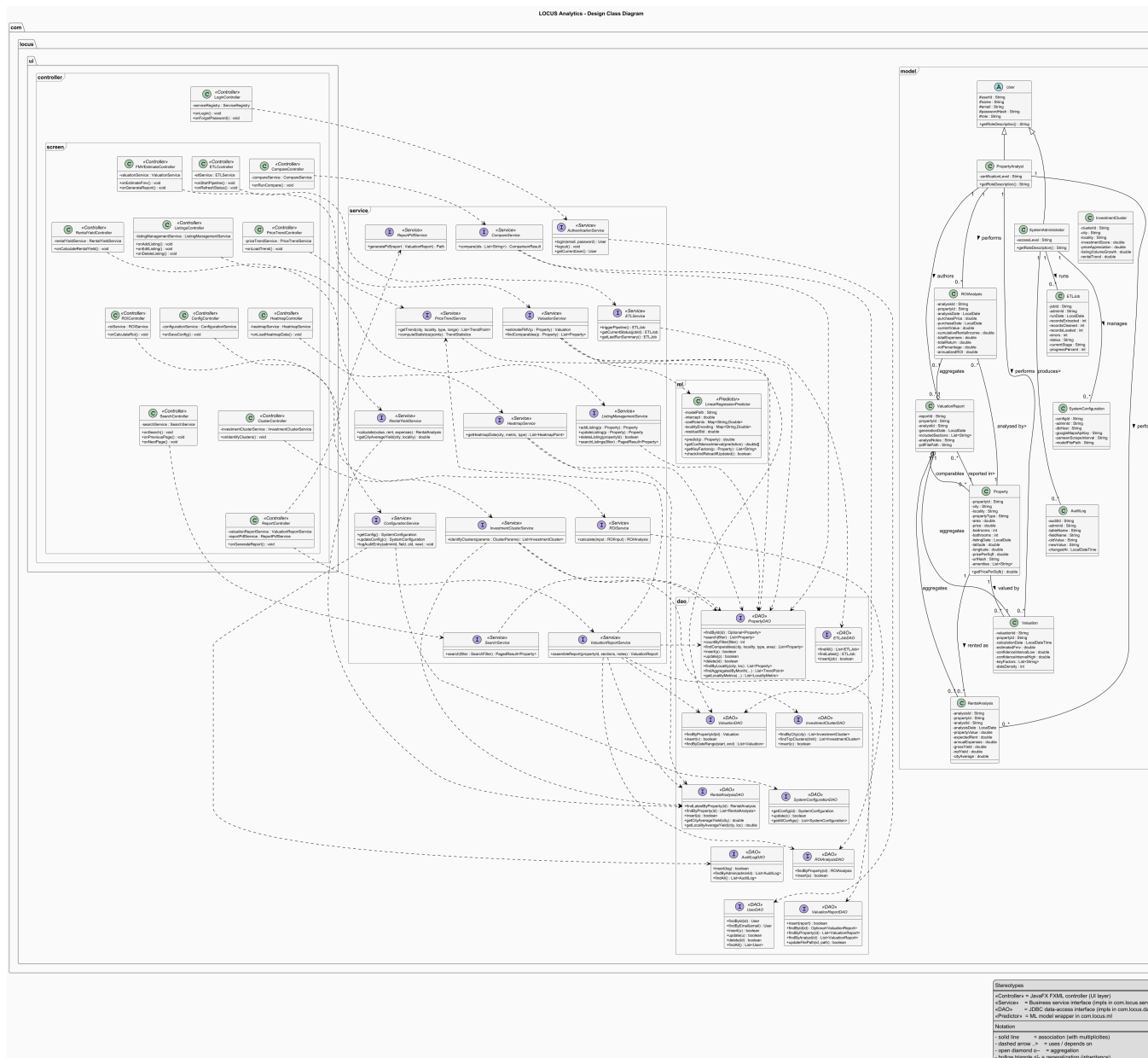
The implementation strictly follows an **Interface + Impl** contract for both the service and DAO layers. The `ServiceRegistry` (`com.locus.ui.ServiceRegistry`) is the single composition root that instantiates one DAO per table and one service per use case, then injects the bundle into each FXML controller.

11.2 Key Design Decisions Reflected in the Diagram

- One controller per screen, not per actor.** The 13 FXML controllers (`LoginController`, `FMVEstimateController`, `RentalYieldController`, `ROIController`, `SearchController`, `CompareController`, `PriceTrendController`, `HeatmapController`, `ClusterController`, `ReportController`, `ETLController`, `ListingsController`, `ConfigController`) each own exactly one screen. Actor → screen mapping lives on the Use Case Diagrams in §8.1, not here.
- Façade services per use case.** Every analyst/admin use case has a dedicated «Service» interface with a narrow API. The `ValuationReportService` is the only service that depends on another service (`PriceTrendService`) — every other service depends purely on DAOs and/or the predictor.
- DAO-per-table.** There is no generic `Repository` abstraction; ten concrete DAO interfaces map 1:1 to the schema tables. `PropertyDAO` is the most-used DAO (six services depend on it) because property data underpins almost every analytical use case.

- ML kept out of the service package.** `LinearRegressionPredictor` lives in its own `com.locus.ml` package and is a stateful pure-fabrication: it is loaded at startup and shared across `ValuationService`. The predictor exposes a reload hook (`checkAndReloadIfUpdated()`), which can be invoked by runtime orchestration paths.
- Entity behaviour is intentionally minimal.** Domain classes are anaemic JavaBeans (getters/setters + a few derived getters such as `Property.getPricePerSqft()`); all calculations live in the matching service. This matches the actual codebase rather than a textbook "rich domain" model.
- Single-table user inheritance.** `User` is abstract; the `role` column in `app_user` discriminates between `PropertyAnalyst` and `SystemAdministrator`. User ownership is explicit for records such as `RentalAnalysis`, `ValuationReport`, `ETLJob`, `SystemConfiguration`, and `AuditLog`; other analytics records are primarily keyed by property/locality.
- Aggregation in `ValuationReport`.** The PDF export is the only entity that composes other entities at runtime — it aggregates exactly one `Valuation`, optional `RentalAnalysis` / `ROIAnalysis`, and zero-or-more comparable `Property` records (transient, not stored as FKs).

11.3 Diagram



Notation. Solid line = association (with multiplicities); dashed arrow $\dots \rightarrow$ = uses / depends on; open diamond $\circ--$ = aggregation; hollow triangle $\triangleleft$ = generalization (inheritance). DTO classes (`SearchFilter`, `PagedResult`, `TrendPoint`, `TrendStatistics`, `HeatmapPoint`, `ROIInput`, `ClusterParams`, `ComparisonResult`, `LocalityMetric`) are referenced by name in service signatures but are omitted as separate boxes for legibility — they live in `com.locus.model.dto`.

11.4 Design Patterns Used

The architecture above is not the result of one big design choice but the application of a small, deliberately chosen catalogue of patterns. This section walks through the patterns that actually appear in the Java source tree (not aspirational ones) and cites the file in which each is realised, so the link between the textbook pattern and the LOCUS implementation is verifiable.

11.4.1 GRASP Patterns (Larman)

GRASP gives us nine general responsibility-assignment principles. The codebase exercises every one of them at least once.

GRASP Pattern	Where it appears in LOCUS	Evidence
Controller	One JavaFX <i>façade controller</i> per use case across <code>com.locus.ui.controller</code> and <code>com.locus.ui.controller.screen</code> (<code>LoginController</code> , <code>FMVEstimateController</code> , <code>RentalYieldController</code> , <code>ROIController</code> , <code>SearchController</code> , <code>CompareController</code> , <code>PriceTrendController</code> , <code>HeatmapController</code> , <code>ClusterController</code> , <code>ReportController</code> , <code>ETLController</code> , <code>ListingsController</code> , <code>ConfigController</code>). Each is the single system event handler for its screen.	Handlers like <code>onEstimateFmv()</code> / <code>onCalculateRentalYield()</code> / <code>onStartPipeline()</code> map to the use-case operations in §9 SSDs.
Information Expert	Domain entities own data they alone are best placed to derive. <code>Property.getPricePerSqft()</code> returns <code>price / area</code> from its own fields; <code>Valuation</code> exposes its own confidence interval bounds; <code>ETLJob.toString()</code> formats its own progress string.	<code>com.locus.model.Property</code> lines 156-162; <code>com.locus.model.Valuation</code> lines 85-99.
Creator	A class creates an object when it aggregates, contains, or has the initialising data for it. <code>ServiceRegistry</code> is the <i>composition root</i> that creates every DAO and service. <code>ETLServiceImpl.triggerPipeline()</code> creates new <code>ETLJob</code> instances; <code>ValuationReportServiceImpl.assembleReport()</code> creates new <code>ValuationReport</code> instances; <code>ValuationServiceImpl.estimateFMV()</code> creates <code>Valuation</code> objects.	<code>com.locus.ui.ServiceRegistry</code> (constructs ten DAO impls and fourteen service impls); <code>ETLServiceImpl:25-33</code> .
Low Coupling	Every cross-layer reference is to an <i>interface</i> , never a concrete class. Controllers hold <code>ValuationService</code> (interface) fields, not <code>ValuationServiceImpl</code> ; services hold <code>PropertyDAO</code> fields, not <code>PropertyDAOImpl</code> . Swapping JDBC for JPA tomorrow would not change a single import in the UI or service code.	All controller fields under <code>com.locus.ui.controller.screen</code> use the interface type; same for every <code>*ServiceImpl</code> constructor.
High Cohesion	One service per use case, with a 1-to-3-method API. <code>ROIService</code> exposes only <code>calculate(ROIInput)</code> ; <code>SearchService</code> only <code>search(SearchFilter)</code> ; <code>HeatmapService</code> only <code>getHeatmapData(...)</code> . No god-classes.	All 14 interfaces in <code>com.locus.service</code> have ≤ 4 public methods.
Polymorphism	Stub and Impl variants exist for most service interfaces, and controllers target interface types rather than concrete classes. This preserves substitutability across test/dev and production implementations.	<code>ValuationServiceStub</code> vs <code>ValuationServiceImpl</code> and similar pairs across <code>com.locus.service.impl</code> (with production wiring currently using Impl services in <code>ServiceRegistry</code>).
Pure Fabrication	Classes that have no domain analogue are <i>fabricated</i> to keep the domain model clean: every <code>*Service</code> , every <code>*DAO</code> , <code>LinearRegressionPredictor</code> , <code>InputValidator</code> , <code>SceneManager</code> , and <code>ServiceRegistry</code> . None of them appears in the conceptual domain model in §6.	<code>com.locus.service.*</code> , <code>com.locus.dao.*</code> , <code>com.locus.ml.LinearRegressionPredictor</code> , <code>com.locus.service.validation.InputValidator</code> , <code>com.locus.ui.SceneManager</code> , <code>com.locus.ui.ServiceRegistry</code> .
Indirection	Two notable indirection layers: (a) services indirect between controllers and DAOs so the UI never speaks JDBC; (b) <code>LinearRegressionPredictor</code> indirections between <code>ValuationService</code> and the on-disk <code>model.json</code> file produced by Python training, so a model swap requires no service change.	<code>ValuationServiceImpl</code> calls <code>predictor.predict(p)</code> and <code>propertyDAO.findComparables(...)</code> — never raw JDBC, never raw JSON.
Protected Variations	Interface boundaries protect the system from likely points of variation: persistence engine (DAO interfaces), ML model format (<code>LinearRegressionPredictor.checkAndReloadIfUpdated()</code>), UI framework (controllers behind FXML), and even the language boundary (Python ETL scraped behind <code>ETLService</code>).	Hot-reload of a new <code>model.json</code> does not require recompiling Java.

11.4.2 Gang-of-Four & Java Enterprise Patterns

Beyond GRASP, the implementation uses several classic GoF patterns and a couple of Java-EE-flavour structural patterns that are well-known in the JDBC community.

Pattern (Category)	Where it appears in LOCUS	Evidence
Façade (GoF — Structural)	Each <code>*Service</code> is a façade that presents a single, use-case-shaped operation while hiding multiple DAO calls, validation, and orchestration behind it. The clearest example is <code>ValuationReportServiceImpl</code> , whose <code>assembleReport(...)</code> orchestrates <code>PropertyDAO</code> , <code>ValuationDAO</code> , <code>RentalAnalysisDAO</code> , <code>ROIAnalysisDAO</code> , <code>ValuationReportDAO</code> , and <code>PriceTrendService</code> .	<code>com.locus.service.impl.ValuationReportServi</code>
Strategy (GoF — Behavioural)	Stub vs Impl pairs provide strategy-style algorithm substitution for many service interfaces (full DB/ML pipeline vs canned in-memory behavior), while FXML controllers remain interface-driven clients.	<code>*ServiceStub</code> / <code>*ServiceImpl</code> pairs in <code>com.locu: ServiceRegistry</code> currently wires Impl services for p
Builder (fluent) (GoF — Creational)	<code>InputValidator</code> accumulates errors via a chained, fluent API: <code>new InputValidator().validateCity(city).validatePropertyType(t).validatePositive("area", a).throwIfInvalid()</code> . Each method returns <code>this</code> , the final call materialises the <code>ValidationException</code> with all collected errors at once.	<code>com.locus.service.validation.InputValidator</code>
Template Method (GoF — Behavioural)	<code>AbstractScreenController.initialize(...)</code> is the template: it sets the screen title and description by calling two abstract hooks <code>getScreenTitle()</code> and <code>getScreenDescription()</code> that every concrete placeholder controller must supply.	<code>com.locus.ui.controller.screen.AbstractScre</code> 38.
Adapter (GoF — Structural)	Two cross-language adapters: (a) <code>LinearRegressionPredictor</code> adapts the JSON Ridge-regression model produced by Python's scikit-learn into a Java <code>predict(Property) → double</code> API; (b) <code>ETLServiceImpl</code> adapts the Python ETL pipeline (launched via <code>ProcessBuilder</code>) into a Java polling API based on <code>ETLJob.progressPercent</code> .	<code>com.locus.ml.LinearRegressionPredictor:120-</code> <code>com.locus.service.impl.ETLServiceImpl:40-86</code>
Observer (GoF — Behavioural)	The ETL Dashboard binds JavaFX <code>DoubleProperty</code> instances (<code>successAnimated</code> , <code>failedAnimated</code> , <code>progressAnimated</code>) to UI labels via <code>addListener(...)</code> , so any update to the property automatically refreshes the screen. The <code>MainController</code> similarly listens for navigation events.	<code>com.locus.ui.controller.screen.ETLControlle</code> <code>addListener</code> calls).
DAO (Java EE Core — Architectural)	A textbook DAO layer (interface + impl, one per table) under <code>com.locus.dao</code> and <code>com.locus.dao.impl</code> , with persistence exceptions surfaced through DAO/runtime boundaries.	<code>com.locus.dao.PropertyDAO</code> + <code>com.locus.dao.impl.PropertyDAOImpl</code> ; DAO impl <code>com.locus.dao.impl</code> .
Data Transfer Object (DTO) (Java EE Core)	A dedicated <code>com.locus.model.dto</code> package carries cross-layer parameter and result objects: <code>SearchFilter</code> , <code>PagedResult<Property></code> , <code>TrendPoint</code> , <code>TrendStatistics</code> , <code>HeatmapPoint</code> , <code>ROIInput</code> , <code>ClusterParams</code> , <code>ComparisonResult</code> , <code>LocalityMetric</code> . They contain no business logic — only data.	<code>com.locus.model.dto.*</code> (10 classes).
Dependency Injection (constructor) (Fowler / DI)	Every service and DAO impl receives its collaborators through its constructor; nothing is looked up via a static singleton or <code>new</code> . <code>ServiceRegistry</code> is the single composition root that performs this wiring at application startup.	<code>ServiceRegistry:50-82</code> (<code>new ValuationServiceImpl(propertyDAO, valuationf</code> etc.).
MVC variant (FXML / Model-View-Controller) (Architectural)	JavaFX's FXML pattern cleanly separates the three roles: FXML files in <code>src/main/resources/fxml/</code> are the View, <code>com.locus.model.*</code> are the Model, and the <code>com.locus.ui.controller.screen.*</code> classes are the Controllers wired via <code>@FXML</code> injection.	Every <code>*View.fxml</code> / <code>*Controller.java</code> pair.

11.4.3 Interactions Between Patterns

A few patterns reinforce each other in LOCUS — they are not used in isolation:

- Façade + DAO + DTO.** Each service is a Façade over multiple DAOs, and the data crossing the boundary is always shaped as a DTO (or an entity), never as a JDBC `ResultSet`. This is what lets the JavaFX UI remain framework-pure.
- Strategy + Dependency Injection.** The Stub/Impl Strategy pair is only useful because `ServiceRegistry` *injects* the chosen implementation; without DI, the controller would have to `new` a class and the substitution would be impossible.
- Adapter + Protected Variations.** `LinearRegressionPredictor` is an Adapter that *implements* the Protected Variations principle: the volatile `model.json` file format is the variation, and the `predict()`-shaped API is the protective seam.
- Template Method + Polymorphism.** `AbstractScreenController` (Template Method) and the Stub/Impl service pair (Polymorphism) together let new screens and new service implementations be added without touching existing call sites.

What was deliberately *not* used. The codebase avoids Singleton (real singletons — `ServiceRegistry` is a *single instance* but is constructed once and passed by reference, not enforced by static state), avoids the Active Record anti-pattern (entities are anaemic JavaBeans, behaviour is in services), and avoids generic `Repository<T>` abstractions in favour of typed DAOs that expose query methods specific to the table they manage.

12. Backend — DAO & Service Layer

The backend follows a clean Interface + Impl contract. Every DAO and every service exposes a Java interface and ships a single implementation. Tests target the interface, allowing in-memory stubs to be substituted without a live database.

12.1 DAO Layer (`com.locus.dao`)

Interface	Responsibility
UserDAO	Authenticate, list, create users; BCrypt verify.
PropertyDAO	CRUD + filtered listing queries with pagination.
ValuationDAO	Persist FMV results; latest-by-property lookups.
RentalAnalysisDAO	Save and fetch yield computations per property.
ROIAnalysisDAO	Persist ROI inputs and computed return metrics.
ValuationReportDAO	Track generated PDF reports + included sections.
InvestmentClusterDAO	Locality-level scoring records and rankings.
ETLJobDAO	Live progress updates for the ETL Dashboard binding.
SystemConfigurationDAO	Settings get / set; audit-trigger surfaces history.
AuditLogDAO	Read-only access to the <code>audit_log</code> timeline.

12.2 Service Layer (`com.locus.service`)

Service	Highlights
ValuationService	FMV via <code>LinearRegressionPredictor</code> + comparables window; CI from residual std.
RentalYieldService	Gross / net yield with annual-rent sanity checks (rent < value).
ROIService	Total return, ROI%, annualised ROI; future-date guard; auto-estimate hook.
SearchService	Filter → paged result; min/max price ordering rule.
CompareService	2 – 4 properties side-by-side with BEST/WORST cell tagging.
PriceTrendService	Monthly aggregation; locality fallback to city when sparse.
HeatmapService	Locality → metric points for Google Maps heatmap layer.
InvestmentClusterService	Composite score: appreciation + volume growth + rental trend.
ValuationReportService	Section orchestration + DAO persistence of report metadata.
ReportPdfService	PDFBox renderer for the branded valuation deliverable.
ETLService	Triggers Python pipeline; binds progress to UI.
ListingManagementService	Admin CRUD for properties with validation.
ConfigurationService	Settings IO; mutations recorded in <code>audit_log</code> .
AuthenticationService	Login + role gating; BCrypt verification.

13. Frontend — JavaFX UI Layer

The UI is FXML-driven, with one FXML per screen and a Controller per FXML. A `SceneManager` coordinates navigation, and `ServiceRegistry` injects the relevant service implementation into each controller. Branding is centralised in `BrandAssets.java` and `styles.css`.

13.1 Screens

FXML	Screen	Audience
LoginView.fxml	Login (BCrypt verify)	All users
IntroView.fxml	Branded intro + media splash	All users
MainView.fxml	Sidebar + content host	All users
screens/FMVEstimateView.fxml	UC-1 FMV Estimate	Analyst / Admin
screens/RentalYieldView.fxml	UC-2 Rental Yield	Analyst / Admin
screens/ROIView.fxml	UC-3 ROI Calculator	Analyst / Admin
screens/SearchView.fxml	UC-4 Search	Analyst / Admin
screens/CompareView.fxml	UC-5 Compare	Analyst / Admin
screens/PriceTrendView.fxml	UC-6 Price Trends	Analyst / Admin
screens/HeatmapView.fxml	UC-7 Heatmap (WebView)	Analyst / Admin
screens/ReportView.fxml	UC-8 Valuation Report	Analyst / Admin
screens/ClusterView.fxml	UC-9 Investment Clusters	Analyst / Admin
screens/ETLView.fxml	UC-10 ETL Dashboard	Admin only
screens/ListingsView.fxml	UC-11 Manage Listings	Admin only
screens/ConfigView.fxml	UC-12 System Configuration	Admin only

13.2 UX Conventions

- **Validation surface:** field-level error styling through CSS pseudo-classes; service-side errors raise modal dialogs.
- **Charts:** JavaFX Charts for trend/yield/ROI breakdowns; pie + line + bar consistent across screens.
- **Maps:** embedded `WebView` loading `heatmap.html` and `clusters-map.html` with property data injected as JSON.
- **Accessibility:** minimum window size enforced; all screens wrap in `ScrollPane` for low-resolution displays.
- **Branding:** dark-green primary, gold accents, Helvetica system font — mirrored in this report.

14. ETL Pipeline

The ETL pipeline is an idempotent Python program that the Java app launches on demand from the ETL Dashboard. It writes progress to the `etl_job` table, which the UI polls to drive the live progress bar.

14.1 Pipeline Stages

Stage	Module	Output
Extract	<code>etl/zameen_scraper.py</code>	Raw listing dictionaries (title, price, area, locality, url).
Clean	<code>cleaner.py</code> / inline transforms in <code>run_etl.py</code>	Type coercion, currency normalisation, locality whitelisting.
Load	<code>etl/run_etl.py</code> → PostgreSQL	Idempotent UPSERT on <code>url_hash</code> ; rejects duplicates.
Audit	<code>etl_job</code> table	<code>records_extracted</code> / <code>cleaned</code> / <code>loaded</code> / <code>errors</code> + <code>status</code> + <code>progress</code> .

14.2 Operational Properties

- **Idempotent:** re-running after a crash skips already-loaded records via `url_hash`.
- **Restart-safe via deduplication:** reruns avoid duplicate inserts through `url_hash` checks and conflict handling.
- **Observable:** live binding to the JavaFX progress bar via DAO polling.
- **Decoupled:** the Python process and Java host share state only through PostgreSQL.
- **Triggers retraining:** after a successful run, `retrain_model.py` can refresh `model.json` without a restart.

15. Machine Learning Module

The intelligence layer is a log-transformed Ridge Regression. Training is offline (Python, scikit-learn); inference is in-process Java reading a serialised model artifact (`ml/model1.json`). The predictor includes a file-change reload hook for runtime integration.

15.1 Training Pipeline

- `train_model.py` reads from PostgreSQL, builds engineered features (locality dummies, type dummies, area, beds, baths), fits a Ridge model on `log(price)`, and writes coefficients to `model.json`.
- `retrain_model.py` performs the same fit on the latest data and compares R^2 against the previous run before promoting the new model.
- `metrics_history.json` persists every training metric over time — a lightweight model registry.

15.2 Accuracy Snapshot (Latest committed metrics)

Metric	Value	Interpretation
R-squared	0.857	Explains 85.7% of price variance on the latest committed run.
MAE	11,299,624	Mean absolute error from latest committed metrics history.
RMSE	25,069,127	Root mean squared error from latest committed metrics history.
Source	<code>ml/metrics_history.json</code>	Snapshot should be refreshed after each retraining cycle.

15.3 Feature Importance

- Locality (0.997):** dominant driver — high-tier areas like DHA Phase 8, Gulberg act as strong multipliers.
- Property type coefficients:** See current `ml/model.json` for exact values (they evolve with retraining data).
- Bathrooms / bedrooms effects:** Present but dataset-dependent; use the current committed model artifact for exact magnitudes.

15.4 Limitations

- Ultra-luxury outliers (>PKR 500M) are sparse and exhibit higher variance.
- Amenity-level features (smart-home, finishes) are only proxied via locality.
- Volatility periods (currency shocks) require re-training to avoid drift.

16. Reporting & PDF Generation

UC-8 (Generate Valuation Report) is implemented by `ReportPdfService` using Apache PDFBox. The report is composable — analysts choose which sections to include (FMV, Comparables, Rental Yield, ROI, Price Trend) and may attach free-form notes. Sections without underlying data render as *Not calculated* stubs rather than failing the export.

16.1 Section Catalogue

Section	Source
Cover & Property Summary	<code>PropertyDAO</code> + analyst metadata
FMV Estimate & Confidence Interval	<code>ValuationService</code>
Comparable Listings	<code>PropertyDAO</code> range query (locality + type)
Rental Yield Analysis	<code>RentalYieldService</code> (latest record)
ROI Analysis	<code>ROIService</code> (latest record)
Price Trend Chart	<code>PriceTrendService</code> (locality → city fallback)
Analyst Notes	<code>ValuationReport.analyst_notes</code>

16.2 Behaviours

- Pre-flight check:** if no FMV exists for the analyst, the export is blocked with a guided message.
- Graceful degradation:** missing rental / ROI sections show an inline notice instead of breaking the layout.
- Persisted record:** every export writes a row in `valuation_report` for traceability.
- Branding:** shared LOCUS palette and logo for visual continuity with the desktop UI.

17. Testing Strategy

Quality assurance combines a broad JUnit unit-test suite at the service layer with a documented manual end-to-end test plan covering UC flows and cross-screen journeys.

17.1 Unit Tests (JUnit Jupiter)

Test Class	Cases	Focus
ValuationServiceTest	FMV	predictor adapter, CI computation, validation
RentalYieldServiceTest	Yield	gross/net math, expense edge cases
ROIServiceTest	ROI	future-date guard, annualisation
SearchServiceTest	Search	filter combinations, pagination
CompareServiceTest	Compare	BEST/WORST cell tagging, 2–4 bound
PriceTrendServiceTest	Trend	monthly aggregation, locality fallback
InvestmentClusterServiceTest	Cluster	score normalisation, ranking
ValuationReportServiceTest	Report	section toggling, persistence

These tests focus on core analytical services and use isolated test doubles/stubs; additional direct coverage for ETL/listings/config/report-pdf/authentication service paths can be added in future hardening passes.

17.2 End-to-End Test Plan

- Login matrix (5 cases) — covers RBAC sidebar gating.
- 12 use-case happy-path + edge-case batteries (FMV-1–6, RY-1–6, ROI-1–6, S-1–8, C-1–6, PT-1–6, HM-1–5, R-1–6, IC-1–6, E-1–5, ML-1–6, SC-1–5).
- Cross-UC flows: Search → Compare → Report; FMV → Yield → ROI → Report; ETL → retrain → FMV.
- RBAC confirmation matrix and 5 error / edge-case scenarios (DB drop, GMaps quota, blank submits, rapid nav, min-size resize).

18. Deployment & Operations

The desktop application ships as a single fat JAR built by the Maven Shade plugin. Setup is automated with documented prerequisites; backups are scripted for both POSIX and Windows hosts.

18.1 Setup Steps (Operator View)

1. Install Java 17, Maven 3.8+, PostgreSQL 15+, Python 3.10+
2. `CREATE DATABASE locus_analytics`
3. `psql -d locus_analytics -f schema.sql`
4. Edit `src/main/resources/config.properties` (DB + GMaps keys)
5. `python data/generate_seed.py` — 800 realistic seed rows
6. `python ml/train_model.py` — produces `ml/model.json`
7. Insert default admin + analyst users (BCrypt-hashed)
8. `mvn clean package -DskipTests`
9. `java -jar target/locus-analytics-1.0-SNAPSHOT.jar`

18.2 Operations

- **Backups:** `scripts/backup.sh` (POSIX) and `backup.bat` (Windows) — weekly rotation, 4-week retention.
- **Scheduling:** see `scheduler_setup.md` for cron / Task Scheduler examples.
- **Hot reload:** Java app watches `ml/model.json` ; retrain triggers an in-place model swap.
- **Auditability:** every config change leaves an `audit_log` row with old / new JSON snapshots.

19. Conclusion

LOCUS Analytics demonstrates a disciplined, end-to-end engineering effort: a layered Java desktop application with strict service / DAO boundaries, a relational schema with indexing and audit trails, an idempotent Python ETL flow, and a Ridge-regression valuation model integrated into the Java service layer.

The codebase includes 12 delivered use cases, 14 FXML screens, service-layer JUnit coverage for core analytics, a manual E2E test plan, scripted backups, and setup documentation. The architecture leaves clear seams for future work — richer feature engineering, additional cities, mobile companion clients, or an authenticated REST facade — without disturbing the present design.

Acknowledgements

Built by **Fasih UI Mubashir** (Backend / DB / ETL), **Arham Manzoor** (Domain / Services / ML), and **Ayaan Aman** (UI / Charts / Maps / PDF) — Bachelor's program, FAST-NUCES.